Quantitative Analysis and Predictive Modeling of Football Performance Metrics

Yarkın Yorulmaz
Student ID: 2797380



Supervisor: Professor Peter Tino

A dissertation submitted to the University of Birmingham
for the degree of MSc in Data Science

School of Computer Science
College of Engineering and Physical Sciences
University of Birmingham

2024–2025

Abstract

This project investigates predictive modeling in professional football by combining raw match statistics, advanced performance metrics, and custom-engineered features. The primary objective was to predict the results of the matches using data-driven methods while addressing the limitations of relying solely on post-match statistics.

Data was collected from multiple sources including Understat, Sofascore, FIFAIndex, and ClubElo, employing web scraping, public API's, and optical character recognition where necessary. A comprehensive data-cleaning pipeline was implemented, including re-scraping to address missing values, systematic checks for multi-collinearity, and normalization of FIFA attributes to capture positional and group-level strengths while reducing redundancy.

A key contribution of this study is the design of a player-level Elo rating system. Unlike traditional Elo implementations, ratings were reset each season, restricted to league matches, and calculated at the player level before aggregation to the team level. Parameter tuning ensured meaningful early-season progression and a more dynamic reflection of player and team contributions.

Match-level features were predicted through rolling aggregation methods, including linear regression, exponential and weighted moving averages, and medians. Models such as Random Forests, Neural Networks, and XGBoost were used to generate predictive features and final outcome classifications. Feature importance analysis was used to refine inputs and highlight the most influential predictors. The results demonstrated that the engineered features, particularly the custom Elo ratings, outperformed many pre-manufactured metrics. Although predicting draws remained a challenge, overall accuracy and macro-F1 scores showed improvements when aggregation and Elo-based features were included.

This work contributes to the growing field of football analytics by presenting a reproducible framework to integrate player-level ratings, advanced feature engineering, and predictive modeling, with potential applications in sports strategy and decision-making.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Peter Tino, for his invaluable guidance, encouragement, and mentorship throughout this project. I am also grateful to my inspector, Dr. Masoumeh, for her constructive feedback and insightful inputs, which helped me strengthen the quality of this thesis.

I would especially like to thank my family for their constant support and belief in me. In particular, I owe a deep debt of gratitude to my father, whose sacrifices made it possible for me to study at this university and complete this work. This thesis would not have been possible without his unwavering dedication.

Use of Generative AI

Generative AI tools (e.g., ChatGPT) were used in a limited and transparent manner during the development of this dissertation. Their role was restricted to improving the clarity and readability of the text by assisting with grammar, phrasing, and structural refinement. They were also used as a support tool for code debugging and restructuring, in the same way that one might use conventional programming forums or documentation.

At no point were these tools used to generate original research content, produce unreferenced material, or bypass the requirement for independent work. All substantive ideas, analysis, experiments, and results presented in this dissertation are my own. Where external sources have been drawn upon, they are properly acknowledged in the bibliography. The use of generative AI therefore served only as an auxiliary aid to improve presentation and technical accuracy, and not as a means of producing or plagiarising academic content.

Abbreviations

ANN	Artificial Neural Network
API	Application Programming Interface
CA	Team-level Elo adjustment factor
CAi	Player-level Elo adjustment factor
DEEP	Deep Completions (passes completed within 20m of goal)
EAi	Expected score of player i in Elo framework
EDA	Exploratory Data Analysis
EMA	Exponential Moving Average
F1	Harmonic mean of precision and recall
FIFA	Fédération Internationale de Football Association (also FIFAIndex ratings dataset)
KPI	Key Performance Indicator
MA	Moving Average
MAE	Mean Absolute Error
ML	Machine Learning
NN	Neural Network
PMI	Player Match Instance
PPDA	Passes Allowed per Defensive Action
RA	Team Elo rating before match
RAi	Player Elo rating before match
R^2	Coefficient of Determination
RF	Random Forest
RM	Rolling Mean
TMI	Team Match Instance
WMA	Weighted Moving Average
xA	Expected Assists
xG	Expected Goals
xGBuildup	Expected Goals Buildup (involvement in buildup play)
xGChain	Expected Goals Chain (involvement in sequences leading to a shot)

Contents

Abstract ii

Acknowledgements iii

Abbreviations iv

List of Figures vii

List of Tables viii

1 Introduction 1
- 1.1 Research Motivation and Limitations . . . 1
- 1.2 Aim and Objectives . . . 1
- 1.3 Thesis Overview . . . 1
- 1.4 Legal, Social, Ethical, and Professional Issues . . . 2

2 Background 3
- 2.1 Football . . . 3
- 2.2 Aggregation Techniques . . . 3
  - 2.2.1 Rolling Mean . . . 3
  - 2.2.2 Rolling Linear Regression . . . 3
  - 2.2.3 Exponential Moving Average (EMA) . . . 3
  - 2.2.4 Weighted Moving Average (WMA) . . . 3
  - 2.2.5 Rolling Median . . . 3
- 2.3 Machine Learning Techniques . . . 4
  - 2.3.1 Random Forest . . . 4
  - 2.3.2 Extreme Gradient Boosting (XGBoost) . . . 4
  - 2.3.3 Artificial Neural Networks (ANNs) . . . 4
- 2.4 Evaluation Metrics . . . 4
  - 2.4.1 Regression Metrics . . . 4
  - 2.4.2 Classification Metrics . . . 4
  - 2.4.3 Confusion Matrix . . . 5
- 2.5 Scraping Methods . . . 5
  - 2.5.1 ScraperFC . . . 5
  - 2.5.2 Selenium . . . 5
- 2.6 Key Metrics . . . 6
  - 2.6.1 Expected Goals (xG) . . . 6
  - 2.6.2 xGBuildup . . . 6
  - 2.6.3 DEEP (Deep Completions) . . . 6
  - 2.6.4 PPDA (Passes Allowed per Defensive Action) . . . 6
  - 2.6.5 Fouled in Final Third . . . 6

3 Literature Review 7
- 3.1 Related Works . . . 7
- 3.2 Research Gap . . . 7

4 Data 9
- 4.1 Understat . . . 9
  - 4.1.1 Data Collection . . . 9
  - 4.1.2 Data Cleaning . . . 9
  - 4.1.3 Indexing and Identifiers . . . 9
  - 4.1.4 Feature Engineering . . . 10
- 4.2 Sofascore . . . 10
  - 4.2.1 Data Collection . . . 11
  - 4.2.2 Data Cleaning . . . 11
  - 4.2.3 Feature Engineering . . . 11
- 4.3 FIFA Dataset . . . 12
  - 4.3.1 Data Collection . . . 12
  - 4.3.2 Data Cleaning . . . 12
  - 4.3.3 Feature Engineering . . . 13
  - 4.3.4 Final Dataset . . . 13
- 4.4 ClubELO and Seasonal Elo . . . 14
  - 4.4.1 Data Collection (ClubELO) . . . 14

4.4.2 Seasonal Elo: Motivation and Framework . . . 14
4.4.3 Seasonal Elo Calculation . . . 15
4.4.4 Rationale . . . 17
4.5 Feature Aggregation . . . 17
4.5.1 Rolling Aggregation . . . 17
4.5.2 Aggregation with Machine Learning . . . 18
4.6 Football-Data.co.uk (Betting Odds) . . . 19
4.6.1 Data Collection . . . 20
4.6.2 Data Cleaning . . . 20
4.6.3 Feature Construction and Baseline Evaluation . . . 20
4.6.4 Output . . . 20
4.7 Merging Data . . . 20
4.7.1 Team Name Matching . . . 20
4.7.2 Player Name Matching . . . 21
4.8 Exploratory Data Analysis (EDA) . . . 22
4.8.1 Seasonal Elo Per Position . . . 22
4.8.2 Game Dominance Across Seasons . . . 23

5 Methodology **24**
5.1 Summary of Pre-processing . . . 24
5.2 Modelling . . . 25
5.2.1 Problem Setup and Splits . . . 25
5.2.2 Feature Families . . . 25
5.2.3 Baselines and Classifiers . . . 25
5.2.4 Evaluation Tracks . . . 26
5.2.5 Feature Selection for Parsimony . . . 26
5.2.6 Pre-result Realization . . . 26
5.2.7 Neural-Network Tuning (AGG→AGG) . . . 27

6 Results and Thoughts **28**
6.1 Master Performance Summary . . . 28
6.2 Neural Network Hyperparameter Tuning (AGG → AGG) . . . 28
6.3 Discussion of Model Performance . . . 28
6.4 Discussion of Feature Importances . . . 29
6.5 Comparison to Betting Odds . . . 29

7 Conclusion **30**

A Appendix **33**
A.1 File branches: . . . 33
A.1.1 Understat . . . 33
A.1.2 Sofascore . . . 33
A.1.3 Football-data-website . . . 33
A.1.4 Fifa . . . 34
A.1.5 ClubELO . . . 34
A.1.6 Aggregation . . . 34
A.1.7 Modeling . . . 34

B Pseudocode Listings **35**

C Tables **39**

List of Figures

4.1 Understat Features (top: roster data/player level; bottom: match data,/team level). [5] 10
4.2 Comparison of Sofascore features: original (top) vs engineered (bottom). [5] 12
4.3 Fifa Features [5] 14
4.4 Best Method Error vs Window Size – Game Dominance 18
4.5 Best Seasonal Elo Ratings Of All Seasons, per Position Played 22
4.6 Comparison of teams with highest and lowest Game Dominance across Premier League seasons. 23

5.1 Workflow [5] 25

B.1 Pseudocode for Sofascore data acquisition and aggregation pipeline 35
B.2 Pseudocode for regression-based imputation and feature gating in data cleaning 36
B.3 Simplified pseudocode for feature aggregation and error analysis pipeline 37
B.4 Simplified pipeline for ML-based feature aggregation 38

List of Tables

4.1 First 15 rows of Mohamed Salah's 2016 Seasonal Elo progression (PMI: Player Match Instance, TMI: Team Match Instance) 15
4.2 Summary of variables used in the Seasonal Elo calculation via Wolf et al.[36] 16
4.3 Example snippet of optimal aggregation method, window size, and mean error for selected features, for Sofascore. 18
4.4 Example Snippet of Comparison of Rolling vs. ML-Based Aggregation Methods(small:top-n features, full:all features) 19
4.5 Final variables from Football-Data.co.uk betting dataset 20
4.6 Example of team name standardisation across datasets. 21

6.1 Test performance across model families and train→test regimes. "Features" indicates the feature set used for training (and testing, unless noted). For proxy experiments, models trained on ORIGINAL are evaluated on AGG/WINDOW proxies. Metrics are on the held-out test set. . . 28
6.2 Neural Network tuning on AGG→AGG (Top-10 features). Metrics are on the held-out test set. 28
6.3 Top-10 most important features for ORIGINAL and AGG→AGG models. 29
6.4 Comparison of betting odds and draw-weighted NN performance on the test set (684 matches). 29

C.1 Condensed Feature Inventory with Full Source/Type Labels 39

Chapter 1
Introduction

Football is one of the most globally followed sports, with billions of fans and vast financial, cultural, and social influence. Predicting match outcomes has long been of interest not only to supporters and analysts, but also to clubs, scouts, and the betting industry. However, accurate prediction is challenging due to the complexity of the game, variability in player performance, and the unpredictability of certain outcomes, particularly draws.

This project explores the application of data science techniques to football prediction. By integrating raw match statistics, pre-manufactured advanced metrics, and custom-engineered features, it aims to build a more robust predictive framework. Data was collected from multiple sources, processed, and merged into a unified dataset, which was then used to train machine learning models for outcome prediction. Special emphasis was placed on player-level Elo ratings and statistical aggregation techniques to capture both individual and team dynamics.

1.1 Research Motivation and Limitations

The motivation for this research arises from the increasing importance of data analytics in football. Clubs, media, and the betting industry all rely on data to gain competitive advantages. Traditional match statistics, however, are insufficient when used in isolation.

The project seeks to overcome these shortcomings by introducing research on the ways to aggregate in-game key metrics engineered features, particularly player-level Elo and official FIFA ratings. These features provide a more nuanced understanding of team and player performance.

Key Limitations

- **Data availability and quality** – Because of the time constraint of which the project was being developed, the data that was able to be gathered was limited in terms of range and depth.
- **Model limitations** – Predicting different classes within competitive sports remains inherently difficult.

1.2 Aim and Objectives

Aim: To predict professional football match outcomes using a combination of in-game, pre-manufactured, and self-engineered features.

Objectives:

- Collect, clean, and integrate football data from multiple sources (Understat, Sofascore, FIFAIndex, ClubElo, Football-data.co.uk).
- Design and implement a player-level Elo rating system with seasonal resets.
- Develop aggregation techniques to generate pre-match predictive features.
- Train and evaluate machine learning models for outcome prediction.
- Analyse feature importance to understand key drivers of performance.
- Identify limitations and propose improvements for future work.

1.3 Thesis Overview

Football matches are decided within a simple but impactful outcome framework: a win awards three points, a draw one point, and a loss none. This points system forms the foundation of league standings and directly influences team success, player careers, and financial rewards. As a result, predicting outcomes is of significant interest not only to fans and analysts, but also to clubs, scouts, and the betting industry.

Betting markets, for example, routinely translate team strength and historical performance into implied probabilities for win, draw, or loss. A bookmaker might express this as odds of 2.0 for a home win, 3.2 for a draw, and 4.5 for an away win, which correspond to estimated probabilities once margins are accounted for.

These probabilities reflect collective market expectations, but they are often shaped by sentiment, popularity, and financial incentives rather than purely by data-driven reasoning.

From an analytical perspective, outcome prediction is especially challenging due to the low-scoring and unpredictable nature of football. Draws are particularly difficult to model, since small random events — a missed penalty, a deflected shot, or a referee decision — can change the result entirely. Unlike sports with high event frequency, such as basketball, the scarcity of goals in football amplifies randomness, making statistical modelling more complex.

This thesis approaches the problem by integrating multiple types of features — raw in-game statistics, pre-manufactured advanced metrics, and custom-engineered predictors — to build models capable of estimating outcome probabilities. In doing so, it aims to bridge the gap between market-based expectations and data-driven performance analysis, contributing to a deeper understanding of how results can be anticipated and explained.

1.4 Legal, Social, Ethical, and Professional Issues

This research was conducted in accordance with legal and ethical standards:

- **Data legality** – Publicly available football data was collected from online sources. Where scraping was used, it was limited to non-sensitive, publicly accessible information.
- **Ethics** – No personal or sensitive data was used. Player statistics are already in the public domain and used strictly for research.
- **Social and professional responsibility** – Predictive football models must be used responsibly. While they have applications in strategy and entertainment, they also intersect with gambling markets, raising concerns about misuse.
- **Bias and fairness** – Care was taken to minimise bias in feature selection and model training. The project also recognises that models cannot fully capture the human and cultural aspects of sport.

Chapter 2
Background

2.1 Football

Football is the world's most popular sport, played by two teams of eleven players over 90 minutes. The objective is simple: score more goals than the opponent. Matches can end in a win, draw, or loss, and league systems typically allocate points accordingly: three points for a win, one point for a draw, and none for a loss. This three-point structure underpins domestic leagues across Europe, such as the English Premier League, La Liga in Spain, and Serie A in Italy.

Each season, teams play one another in home-and-away fixtures, with the champion decided by the total points accumulated. Draws are particularly influential in determining league standings, yet are also the most difficult outcome to predict due to the sport's low-scoring and often unpredictable nature [11].

The statistical complexity of football has attracted significant attention from analysts, clubs, and the betting industry. Bookmakers publish outcome probabilities through betting odds, while clubs and analysts increasingly rely on advanced statistics to inform tactical and strategic decisions. However, predicting match outcomes remains challenging, as performance depends not only on raw statistics, but also on team dynamics, form, and contextual factors such as match importance [3, 9].

2.2 Aggregation Techniques

When predicting football match outcomes, one major challenge is that many in-game statistics are only available after a match has been played. To use such features for pre-match predictions, historical performance needs to be aggregated in a way that captures team form and player trends while smoothing out match-to-match variability. Several aggregation techniques are commonly used for this purpose:

2.2.1 Rolling Mean

The rolling mean, or moving average, calculates the average of a variable across a fixed number of previous matches. This technique smooths out short-term fluctuations and provides a simple measure of recent form. However, it treats all matches in the window equally, which may limit its sensitivity to sudden changes in performance.

2.2.2 Rolling Linear Regression

Rolling linear regression fits a linear model to a sequence of past observations within a fixed window. Unlike the rolling mean, this method captures the trend (increasing or decreasing performance) rather than just the central tendency. It is useful when a team's performance is changing consistently over time, but may be less effective in highly volatile contexts.

2.2.3 Exponential Moving Average (EMA)

The EMA assigns exponentially decreasing weights to older matches, giving more importance to recent performances. This makes it more responsive to sudden changes in form while still retaining information from earlier matches.

2.2.4 Weighted Moving Average (WMA)

The WMA applies custom-defined weights to matches in the aggregation window, allowing analysts to emphasise certain matches (e.g., more recent games or matches of higher importance). Unlike the EMA, the weights are not exponential but chosen explicitly, providing flexibility to tailor the aggregation to specific analytical needs.

2.2.5 Rolling Median

The rolling median calculates the middle value of past observations within a window. It is more robust to outliers than the mean and can provide a stable measure of central tendency, especially in statistics that exhibit extreme variability (e.g., red cards or penalties).

2.3 Machine Learning Techniques

Machine learning has become an essential tool in football analytics, enabling structured data to be transformed into predictive insights [3]. This project primarily applies Random Forests, XGBoost, and Artificial Neural Networks (ANNs). These models are used both for feature prediction (generating pre-match estimates of in-game metrics such as xG or PPDA) and for outcome classification (predicting win, draw, or loss).

2.3.1 Random Forest

Random Forest is an ensemble method that combines multiple decision trees to improve robustness and reduce variance. It can be applied to both regression and classification tasks, making it suitable for predicting continuous features (e.g., shots, passes, possession ratios) as well as categorical outcomes (win/draw/loss). In this project, Random Forests are also used to extract feature importances, identifying which variables contribute most strongly to prediction performance. Implementations are via `scikit-learn` [22].

2.3.2 Extreme Gradient Boosting (XGBoost)

XGBoost is a gradient-boosted tree algorithm designed for scalability and efficiency [7]. It is effective at capturing non-linear relationships in structured data. In this project, XGBoost is used in both regression and classification settings, providing strong baselines for feature aggregation tasks as well as for multiclass outcome prediction. Its regularisation mechanisms help control overfitting, which is important when dealing with highly correlated football features.

2.3.3 Artificial Neural Networks (ANNs)

ANNs mimic the architecture of biological neural systems. They consist of input, hidden, and output layers where weights are updated through backpropagation. ANNs are especially useful for learning complex, non-linear interactions between football variables. In this project, they are used alongside tree-based models to predict feature aggregations and to compare predictive performance in outcome classification tasks. Implementations are via TensorFlow [1].

2.4 Evaluation Metrics

This project involves two distinct predictive tasks: (i) regression tasks for estimating pre-match features, and (ii) multiclass classification for predicting match outcomes (win, draw, or loss). Each task requires tailored evaluation metrics.

2.4.1 Regression Metrics

For regression tasks, such as predicting aggregated features (e.g., rolling xG, PPDA, or shot ratios), the following metrics were used:

Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

MAE measures the average absolute difference between predicted and actual values. It provides an intuitive measure of the typical error magnitude, where lower scores indicate greater accuracy.

Coefficient of Determination (R^2 Score)

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

The R^2 score reflects the proportion of variance in the dependent variable explained by the model. An R^2 of 1 indicates perfect prediction, while 0 suggests the model is no better than predicting the mean.

These metrics were particularly important when testing different rolling aggregation techniques (linear regression, EMA, WMA, median) to determine which provided the most reliable estimates of pre-match statistics.

2.4.2 Classification Metrics

For the main predictive task — multiclass classification of match outcomes (Win, Draw, Loss) — multiple metrics were employed to provide a balanced assessment of model performance.

Accuracy The proportion of correctly classified outcomes across all matches. While intuitive, accuracy can be misleading when classes are imbalanced, as in football where draws occur less frequently.

Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall (Sensitivity)

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1 Score

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Macro-F1 Score In multiclass settings, Macro-F1 calculates the F1 score independently for each class (Win, Draw, Loss) and then averages them. This ensures that each outcome is weighted equally, regardless of class frequency. It was the primary metric for assessing fairness of predictions, since draws were otherwise underrepresented.

2.4.3 Confusion Matrix

To visualise and interpret classification results, a confusion matrix was used. This matrix compares predicted outcomes against actual outcomes in a table format:

	Predicted Win	Predicted Draw	Predicted Loss
Actual Win	True Positive (Win)	False Negative (Draw)	False Negative (Loss)
Actual Draw	False Positive (Win)	True Positive (Draw)	False Negative (Loss)
Actual Loss	False Positive (Win)	False Positive (Draw)	True Positive (Loss)

Each cell in the matrix provides insight into where the model performs well or struggles. For example, high values in the diagonal cells (Win→Win, Draw→Draw, Loss→Loss) indicate correct predictions, while off-diagonal values highlight misclassifications. In this project, confusion matrices revealed that draws were consistently the hardest class to predict, with models often misclassifying them as wins or losses, consistent with prior findings on dependence and low scoring [11].

2.5 Scraping Methods

2.5.1 ScraperFC

ScraperFC is an open-source Python package designed to simplify the extraction of football data from various online sources. The library provides pre-built scraping functions for fixtures, results, player statistics, and betting odds, enabling analysts and researchers to collect data without building scrapers from scratch [26].

Although ScraperFC is limited in scope compared to dedicated analytics platforms such as Understat or Sofascore, it offers a flexible and efficient way to access structured football data. In this project, ScraperFC was occasionally used to supplement the primary scraping pipelines, particularly when quick access to fixture or match data was required. Its inclusion highlights the role of open-source tools in bridging gaps between proprietary databases and research applications.

2.5.2 Selenium

For web scraping, Selenium was used to mimic the behaviour of the Chrome browser and automate interactions with websites. This approach allowed the retrieval of dynamic content that would not be accessible through simple HTML requests. To reduce the likelihood of being flagged as a bot, the Selenium Stealth variant was employed, which modifies browser fingerprints and simulates genuine user activity. This ensured more reliable access to the required datasets and minimised interruptions from anti-bot protections. [27]

2.6 Key Metrics

2.6.1 Expected Goals (xG)

xG estimates the probability of a shot resulting in a goal based on contextual factors such as shot location, shooting angle, assist type, and body part used. It provides a measure of chance quality rather than raw shot counts [4, 34].

2.6.2 xGBuildup

xGBuildup quantifies player involvement in possession sequences that culminate in a shot, excluding the final shot and the assist. This metric highlights a player's contribution to the buildup phase of attacking play [4, 34].

2.6.3 DEEP (Deep Completions)

DEEP counts completed passes received within 20 metres of the opponent's goal, excluding set pieces. It reflects a team or player's ability to penetrate into dangerous attacking zones [34].

2.6.4 PPDA (Passes Allowed per Defensive Action)

PPDA measures pressing intensity by calculating the number of opponent passes allowed before a defensive action (tackles, interceptions, fouls) is attempted. Lower values indicate stronger and more aggressive pressing [32, 34].

2.6.5 Fouled in Final Third

This metric records the number of fouls won by a team or player in the attacking third of the pitch. It is an indicator of offensive pressure, dribbling threat, and the ability to create set-piece opportunities in advanced areas [29].

Chapter 3
Literature Review

3.1 Related Works

The application of statistical modelling and machine learning in football has received increasing attention in recent years, particularly in the context of team and player evaluation, performance analysis, and match outcome prediction. One of the most established approaches is the Elo rating system, originally developed for chess but later adapted to football. Hvattum and Arntzen [15] demonstrated the usefulness of Elo-derived covariates for forecasting football match results, comparing their predictive value against bookmaker odds. Similarly, Wolf et al. [36] proposed a football-specific Elo-based player rating system, updating individual player ratings from line-ups and minutes played. Their results indicated that player-level Elo can provide valuable insights for both team evaluation and outcome prediction. These works underscore the potential of Elo-based approaches but generally focus either on team-level or continuous cross-season ratings.

Alternative rating systems have also emerged. Schultze and Wellbrock [25] introduced a weighted plus–minus model adapted from basketball and hockey, designed to address football's lower scoring rates and substitution constraints. Kharrat et al. [16] expanded this analysis across multiple leagues, confirming the value of adjusted plus–minus models for individual player evaluation. Similarly, Pappalardo et al. [20] developed *PlayeRank*, a role-aware, multidimensional evaluation framework that leverages event-level data to assess player performance in different tactical contexts. These approaches focus on individual contribution and tactical analysis rather than direct outcome prediction.

Another important strand of research uses FIFA player ratings as a data source. Yeung et al. [37] constructed an interpretable match prediction framework that incorporated team formations and grouped FIFA attributes, achieving improved F1 scores compared to baseline odds across five seasons of the English Premier League. Their work highlighted the utility of FIFA ratings for interpretable forecasting, though it did not combine these ratings with dynamic Elo-style models. Related work by Matano et al. [19] proposed using FIFA ratings as prior information to stabilise adjusted plus–minus models, particularly in sparse datasets. These studies show that FIFA-derived attributes can be valuable predictors but also raise issues of collinearity and redundancy that must be addressed when integrating them into predictive pipelines.

More traditional approaches to football prediction include Poisson regression models, first applied by Maher [18], and subsequently refined by Dixon and Coles [11] to account for correlation and recency effects. Bayesian hierarchical models, such as those proposed by Baio and Blangiardo [2], further capture dynamic team strengths over time. While effective for team-level forecasting, these models typically abstract away individual player contributions and rarely integrate external data sources such as FIFA ratings.

Finally, dynamic paired-comparison models, including extensions of the Bradley–Terry framework, offer formal alternatives to Elo updates, explicitly modelling ties and time-varying strengths [6, 17]. These methods provide theoretical generalisations of rating systems but have yet to be widely adopted in applied football analytics.

Overall, the literature reflects three major strands:

1. rating systems such as Elo, plus–minus, and PlayeRank for evaluating player and team strength,
2. FIFA-based studies that use subjective scouting-derived attributes for predictive modelling, and
3. statistical models of goals and outcomes using Poisson or Bayesian frameworks.

Each stream contributes to understanding performance and outcomes, but rarely are they integrated into a unified predictive system.

3.2 Research Gap

Although the literature on football analytics is extensive, there remain several gaps that this dissertation seeks to address.

First, most studies emphasise one dimension of prediction—such as Elo-based ratings, FIFA-derived qualities, or statistical models of match events—without attempting to integrate these complementary perspectives. By combining player-level and seasonal Elo ratings, FIFA-derived positional and skill-group features, and predicted pre-match team statistics, this research offers a unified framework for outcome prediction. This contrasts with existing studies such as Yeung et al. [37], which combine FIFA ratings with formations but omit dynamic

player-level ratings, or Wolf et al. [36], which develop player Elo ratings without incorporating FIFA or aggregated statistical predictors.

Second, existing Elo-based models typically treat ratings as continuous across seasons and competitions. This dissertation introduces a seasonal, league-only Elo rating system at the player level, aggregated into team ratings. Resetting Elo at the beginning of each season and calibrating update factors such as home advantage and responsiveness allows the model to reflect rapid shifts in form and mitigate early-season instability. This represents a methodological innovation compared to conventional Elo systems.

Third, while many studies incorporate realised post-match statistics, they do not address the problem of predicting pre-match feature values. In this dissertation, in-game features such as PPDA [32] or shot conversion ratios are first estimated using rolling aggregation techniques, including exponential moving averages, weighted moving averages, linear regression, and medians. These methods are evaluated using Mean Absolute Error (MAE) and R^2 before being used as inputs to the outcome classification models. This predict-then-classify pipeline directly addresses the temporal challenge of making predictions prior to matches.

Fourth, where FIFA ratings have been employed, there has been limited attention to systematic normalisation or correlation management. This dissertation introduces positional and group-level normalisation to highlight relative strengths and weaknesses, while also removing redundant overall attributes to mitigate collinearity. This allows FIFA ratings to be included in models in a principled and interpretable manner.

Finally, outcome prediction in football is typically framed as a binary classification problem (win vs. loss), whereas this project treats it as a multiclass classification task (win, draw, loss). This distinction is crucial, as draws are a common but notoriously difficult outcome to model. Models in this project are evaluated not only using overall accuracy but also with class-sensitive metrics, including macro-F1, precision, recall, and F1-score, alongside confusion matrix analysis to identify systematic misclassifications. This explicitly addresses the imbalance and difficulty of predicting draws, a limitation noted across the literature [24].

In summary, this dissertation contributes to football analytics by combining three previously separate areas—player-level seasonal Elo ratings, FIFA-derived features, and aggregated pre-match statistics—into a single predictive framework. By employing machine learning methods such as Random Forests [22], XGBoost [7], and Artificial Neural Networks [1] for both feature prediction and outcome classification, it provides a novel approach to forecasting football matches that is both methodologically rigorous and practically relevant.

Beyond outcome-focused rating and regression models, recent research has also leveraged event-level and spatio-temporal datasets to capture the probabilistic value of individual player actions. Pappalardo et al. [21] introduced a large-scale open dataset of match events, while Decroos et al. [10] developed action-based valuation models that quantify player contributions beyond goals and assists. Expected goals (xG) models have also become central to modern football analytics, with early work by Van Haaren and Van den Broeck [35] demonstrating the use of relational learning for match prediction, and practitioners such as Caley [4] contributing widely used xG frameworks. In parallel, machine learning surveys such as Bunker and Susnjak [3] highlight the breadth of algorithms applied to match result prediction, while Constantinou et al. [9] proposed Bayesian networks for probabilistic forecasting. Importantly, Rue and Salvesen [24] noted the persistent difficulty of accurately predicting draws, a challenge that remains evident across predictive methodologies.

Chapter 4
Data

This chapter introduces the datasets used in the thesis. These datasets contain explanatory information about matches, teams, and players, as well as engineered features designed to train the main match prediction model. Data from five primary sources were collected and combined to achieve the objectives of the study. However, data collection itself posed a considerable challenge, as most of the required information was not readily available in downloadable format. Instead, it had to be scraped from the respective websites, which added complexity to the process of merging the datasets. The methods used to collect and process the data are described in the following sections, alongside a detailed account of the datasets themselves and their role within the project. The data collected and used in this project are limited to English Premier League matches and only include match history between the 2015/2016 season and the 2023/2024 season.[34, 29, 12, 8, 14]

The methods by which the data are treated in this thesis are divided into three categories:

- **In-game metrics:** Statistical values directly gathered from player actions during matches, such as shots, passes, fouls, or possession. These represent the raw descriptive elements of match performance.
- **Pre-manufactured metrics:** Features that have already been calculated and published by external providers, such as expected goals (xG), xGBuildup, or ClubELO ratings. These metrics encapsulate more advanced analytical concepts but are not directly observable in the raw match data.
- **Self-generated features:** Features engineered within this project using algorithms or logical transformations. Examples include ratio-based efficiency measures, rolling or weighted aggregations to capture form, and player-level Elo ratings that were aggregated to the team level.

A quick summary of how the datasets were processed and shaped:

- **Match:** Team level data, every row represents a single match between two teams. Found in datasets from sources: Sofascore, Understat, ClubELO, Football-Data.co.uk
- **Roster:** Player level data, every row represents a player, either their performance metrics for a match(Understat, Seasonal Elo) or their generalized seasonal ratings, (FIFA Index).

4.1 Understat

Understat is a football analytics platform that provides both team- and match-level data as well as roster-level player data for each match. In addition to raw statistics, it also supplies advanced performance metrics such as Expected Goals (xG), DEEP, xGChain, and PPDA[4, 21]. Understat was included in this project because it complements Sofascore by not only covering team and match statistics, but also extending to player-level performance per match, thereby allowing for aggregation at both the player and team levels.

4.1.1 Data Collection

Understat integrated well with the built-in ScraperFC functions for data scraping. With only minor adjustments to the scraping loop, the data were gathered into workable DataFrames without any missing values. The resulting dataset consisted of two parts: one containing team-level match data and the other containing player-level roster data for each match.

4.1.2 Data Cleaning

Some features contained values expressed as percentages. These values were converted into float representations by removing the percentage symbol, converting the strings into numeric values, and dividing by 100. Fortunately, the scraping algorithm did not produce any NaN values.

4.1.3 Indexing and Identifiers

Since the Understat datasets include both player-level and team/match-level information, a consistent system of primary keys was established for matches, teams, and players. Identifiers such as team names, player names, and match dates were always standardised to their Understat representations, ensuring the datasets remained organised and consistent.

4.1.4 Feature Engineering

The only engineered feature within this dataset was the consolidation of player-level shots and shots on target in order to reduce dimensionality. It should be noted that some features overlapped between Understat and Sofascore. For example, both datasets contained information on shots and shots on target, but Sofascore provided these values at the match/team level. During aggregation, it was found that Sofascore yielded a lower Mean Absolute Error (MAE), which will be discussed in later sections. Consequently, the Understat version of these features was dropped after combining all datasets.

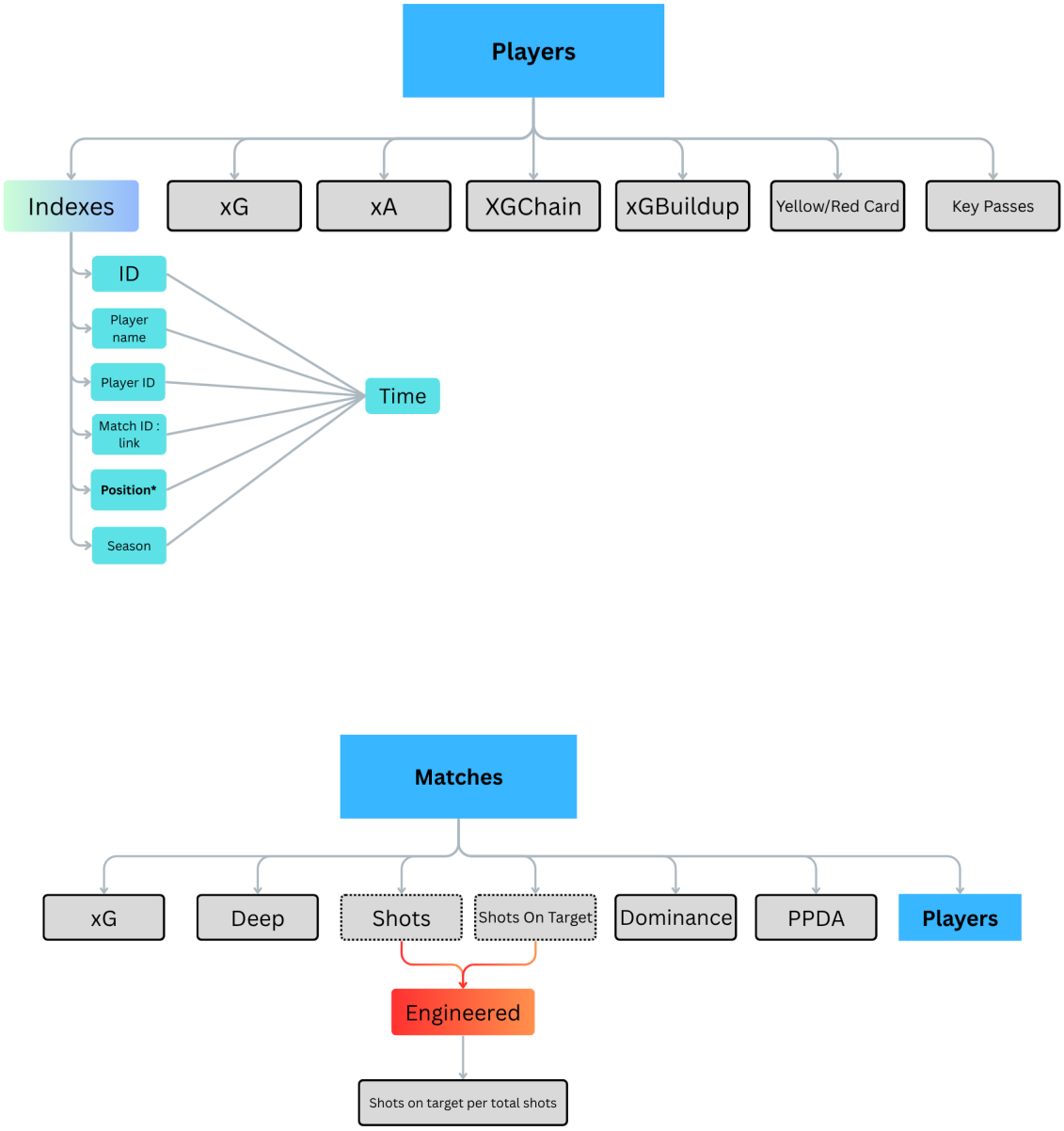


Figure 4.1: Understat Features (top: roster data/player level; bottom: match data,/team level). [5]

4.2 Sofascore

Sofascore is a live sports data platform that provides detailed team/match-level statistics, such as possession, passes, shots, fouls, and disciplinary records. Within the scope of this project, Sofascore was used exclusively for team and match-level data. It was chosen because it offers a comprehensive set of raw explanatory variables that describe the flow and outcomes of matches beyond final scores.[29]

4.2.1 Data Collection

Data collection from Sofascore involved several stages. Initially, the ScraperFC library was used to gather datasets from the platform[26]. However, this approach produced frequent connection errors and resulted in a high proportion of null values, making it unsuitable for large-scale collection. As a result, ScraperFC was employed only for retrieving the necessary match ID values within the specified timeframes. These IDs were then used to construct match links required for scraping the relevant data.

To extract match data from these links, the Selenium module was utilised to mimic a Chrome browser session, with the stealth variant applied to bypass anti-bot detection mechanisms. This ensured more stable scraping when accessing dynamic content. Fortunately, Sofascore also provides a public API, which simplified the process considerably. Once the scraping algorithms were configured, the API responses returned structured JSON dictionaries. These could be indexed into their component branches and directly transformed into Pandas DataFrames for further analysis.

The pipeline of which can be found in pseudocode in (Appendix B.1)

4.2.2 Data Cleaning

A substantial proportion of features contained missing values (NaN) due to several causes: (i) name mismatches arising from Sofascore's evolving statistic labels, (ii) feature non-applicability for certain matches, and (iii) transient connection errors during collection.

To ensure modelling integrity, we applied a systematic cleaning policy:

1. **Feature exclusion threshold.** Features with more than 20% missingness were dropped. This aligns with the planned 80/20 train–test split and avoids excessive imputation.

2. **Type normalisation.** Mixed representations (e.g., fractions and percentages such as `35/62 (56%)` for ground duels) were standardised into numeric integers/floats to ensure compatibility with downstream models.

3. **Regression-based imputation.** For each retained feature with NaNs, we trained polynomial regression models (degrees 1–3) using the remaining available features as predictors. Model selection was based on the minimum validation MAE[22]. The best model was used to impute missing values in the target feature.

4. **Quality gating.** If the imputation error for a feature was unacceptably high (relative to that feature's scale, mean, and standard deviation), the feature was excluded from the final dataset.

The cleaning process pseudocode can be found in (Appendix B.2)

4.2.3 Feature Engineering

To improve the predictive quality of the dataset, raw Sofascore features were first examined for redundancy and correlation. Many of the original statistics were either overlapping in meaning or required contextualisation. Using them in their raw form would introduce multicollinearity, inflate feature space, and reduce interpretability.

To address this, new engineered features were created as ratios or composite measures that expressed relative performance rather than absolute values:

- **Efficiency-based metrics:** Ratios such as blocked shots per shot on target, shots on target per total shots, and hit woodwork per total shots were generated. These capture how effective teams were in turning attempts into meaningful outcomes, providing a clearer signal than raw counts.

- **Shot distribution metrics:** Ratios of shots inside box per total shots and shots outside box per total shots were included to highlight attacking style, differentiating between teams that rely on close-range chances versus long-range attempts.

- **Passing efficiency:** Pass accuracy was derived as accurate passes divided by total passes, reflecting control and quality of possession rather than just volume.

- **Composite dominance metric:** A new feature, game dominance, was engineered by multiplying ball possession with pass accuracy. This aimed to capture not only how long a team held the ball but also the effectiveness of their passing during that possession. Finally, an overall game dominance measure was derived by taking the difference between the home and away dominance scores, providing a single indicator of which side controlled the match.

After these engineered features were created, the raw components were dropped to avoid redundancy. Additional features deemed less informative or highly correlated with stronger predictors were also excluded.

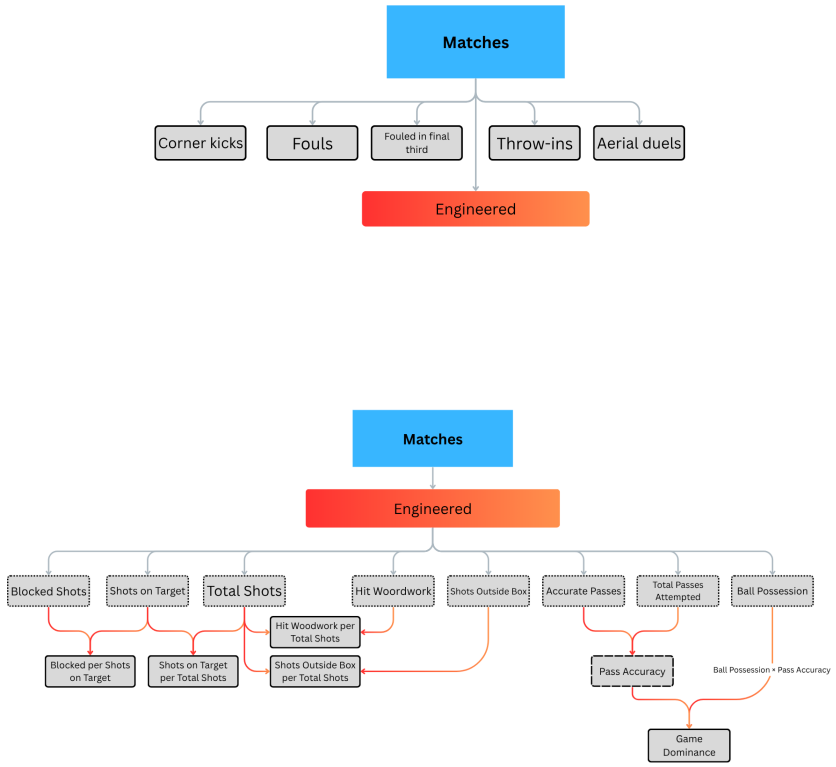


Figure 4.2: Comparison of Sofascore features: original (top) vs engineered (bottom). [5]

4.3 FIFA Dataset

The FIFA dataset was sourced from *FIFAIndex.com*[12], a platform that archives player ratings and detailed attribute values across annual FIFA game releases. Unlike the Understat and Sofascore datasets, which are based on real-world match events, FIFA provides expert- and algorithm-driven estimates of player skills such as passing, shooting, dribbling, and physicality. Including FIFA ratings in this project offered a complementary perspective by supplying stable, player-level performance indicators [30, 19] that are not subject to the noise of individual matches, but instead reflect longer-term assessments of ability.

4.3.1 Data Collection

Player-level data were scraped from FIFAIndex across all editions between FIFA 16 and FIFA 24 for Premier League clubs, and the player ratings change every season. Selenium was used to mimic Chrome browser sessions, with the stealth variant applied to bypass anti-bot detection. A layered scraping pipeline was implemented:

1. **Team URLs:** Each season's Premier League team pages were retrieved to collect links to team rosters.
2. **Player URLs:** From each team roster, individual player profile links were extracted.
3. **Player attributes:** For every player, position-specific cards (Ball Skills, Passing, Shooting, Defence, Physical, Mental, Goalkeeping) were parsed, producing detailed attribute tables.

A retry mechanism with logging was embedded at every stage to ensure resilience against connection errors. Intermediate checkpoints were periodically saved (e.g., `all_teams.csv`, `all_players.csv`, `all_player_stats.csv`) to prevent data loss in case of interruptions.

4.3.2 Data Cleaning

The raw scraped data required extensive cleaning due to inconsistent formatting and occasional missing values:

- **Duplicate and mislabelled columns:** Some features contained numeric suffixes (e.g., "Ball Control 1", "Stamina 6"). These were standardised to canonical names. Erroneous columns (e.g., "Sprint Speed 1", "Stamina 6") were dropped.
- **Version consistency:** Rows without explicit season identifiers (e.g., "fifa24") were corrected to match the expected naming scheme.
- **Team identifiers:** The `Team` field was renamed to `Team Name` to align with other datasets, and player/team names were standardised.
- **Missing values:** Players with incomplete attributes were re-scraped individually. For example, Jérémy Doku (Manchester City, FIFA 24) was specifically added after a missing record was detected.

Finally, player positions from the Understat dataset were cross-referenced to adjust attributes:

- Non-goalkeepers had all goalkeeping attributes set to zero.
- Goalkeepers had outfield attributes set to zero, with minutes set to 90 if they played.

4.3.3 Feature Engineering

Several transformations were applied to convert the raw attributes into interpretable features for modelling:

- **Overall categories:** Composite ratings were calculated for major skill groups, such as:
  - Overall Ball Skills (e.g., dribbling, ball control),
  - Overall Passing (short pass, long pass, crossing),
  - Overall Shooting (finishing, shot power, long shots, penalties),
  - Overall Defence (stand tackle, interceptions),
  - Overall Physical (strength, stamina, balance, speed),
  - Overall Goalkeeping (handling, kicking, positioning, reflexes),
  - Overall Mental (vision, reactions, positioning, aggression).

  A global Overall score was then calculated as the mean of all category ratings.
- **Adjusted attributes:** To reduce collinearity and provide relative measures, "adjusted" versions of individual stats were derived by subtracting the relevant overall category score (e.g., `Finishing_adj = Finishing − Overall Shooting`). This highlighted whether a player was above or below their positional group average.
- **Dimensionality reduction:** Strongly correlated or redundant features (identified via correlation analysis and VIF diagnostics) were dropped, including `Volleys_adj`, `Crossing_adj`, and `Slide Tackle_adj`. Goalkeeping reflexes and diving were merged into a composite GK Save Ability, which was then integrated into the Overall Goalkeeping score.
- **Team-level aggregation:** After cleaning, player attributes were aggregated at the team-match level using minutes-weighted averages for outfield players and mean values for goalkeepers. These aggregated FIFA ratings were then merged with Understat match IDs, producing team-level FIFA-derived features aligned with real-world fixtures.

4.3.4 Final Dataset

The final FIFA dataset contained fully cleaned and engineered player attributes, which represents players **ratings for that respective season.**

It should be noted that the reasoning behind scraping FIFA datasets from **FIFA16** to **FIFA24**, while the other datasets cover the seasons between 2015 and 2023, is due to the way FIFA ratings are constructed. Each FIFA edition evaluates player attributes based on performances from the preceding season. For example, FIFA16 ratings correspond to the 2015/2016 season and so on.

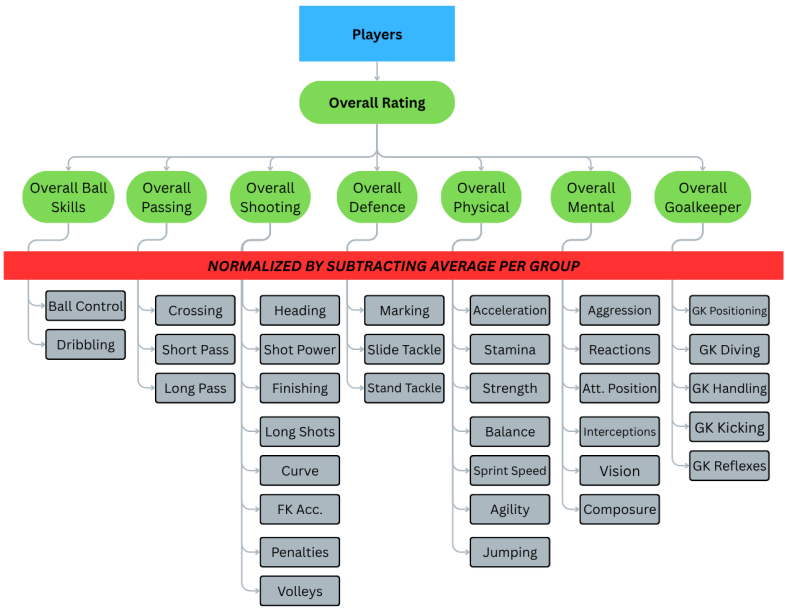


Figure 4.3: Fifa Features [5]

4.4 ClubELO and Seasonal Elo

Elo ratings provide a dynamic estimate of team and player strength, originally developed for chess and later adapted to football[15, 17]. The central principle of Elo is that changes in a team's rating after a match depend not only on the result (win, draw, or loss), but also on the relative strength of the opponent, the goal difference, and contextual factors such as home advantage. In football analytics, Elo ratings have been widely used to evaluate team quality, forecast match outcomes, and benchmark performance trajectories across seasons. This project incorporated two distinct Elo-based measures: ClubELO, an externally maintained dataset, and Seasonal Elo, a custom implementation at both team and player levels.

4.4.1 Data Collection (ClubELO)

Daily team-level Elo ratings were retrieved from the ClubELO API[8]. Using the full set of match dates from the Understat dataset (2015/16–2023/24), HTTP requests were made for each unique date. Responses were provided in CSV format, which were compiled into a consolidated dataset (`clubelo_team_date_elo.csv`). Each record contained the Elo rating of all clubs active on that date, along with their country, competition, and metadata.

To align the two datasets, club names in ClubELO were normalised to match those in Understat (e.g., "Man City" → "Manchester City", "Wolves" → "Wolverhampton Wanderers"). After harmonisation, home and away team Elo values were merged into the Understat match dataset, producing `Elo_team_h` and `Elo_team_a` columns.

4.4.2 Seasonal Elo: Motivation and Framework

While ClubELO captures global, long-term team quality, it does not directly account for seasonal form or player-level contributions. To address this, a Seasonal Elo system was constructed, designed to be sensitive to the performance of teams and individual players within a given season. The approach extended the player-level Elo methodology proposed by Wolf et al. [36], with adaptations to incorporate dynamic update factors and seasonal resets.

- minutes played as a scaling factor for individual influence,
- dynamic k_{Ai} and q_{Ai} parameters that adapt based on player involvement and match recency,
- goal difference weighting, where Elo change scales with the margin of victory or defeat,

The aim is to reset ELO to its initial value every season, and to capture team performances on a player basis. The entirety of the algorithms were manufactured with different loops that calculate target variables row by row.

Table 4.1: First 15 rows of Mohamed Salah's 2016 Seasonal Elo progression (PMI: Player Match Instance, TMI: Team Match Instance)

PMI	TMI	qAi	kAi	EAi	CAi	RAi	CA	RA	RB	New RA	New RAi
1	1	1.000	40.00	0.500000	0.500000	1500.00	0.500000	1500.00	1500.00	1509.25	1510.00
2	2	0.975	39.75	0.515181	0.610833	1510.00	0.617120	1509.25	1499.45	1518.84	1521.48
3	3	0.950	39.50	0.505136	0.623490	1521.48	0.636179	1518.84	1517.91	1530.29	1533.06
4	4	0.925	39.25	0.512081	-0.875646	1533.06	-0.839990	1530.29	1524.66	1508.05	1517.54
5	5	0.900	39.00	0.481865	0.018135	1517.54	0.020899	1508.05	1530.15	1512.16	1517.87
6	6	0.875	38.75	0.577085	0.422915	1517.87	0.432472	1512.16	1463.88	1522.50	1523.94
7	7	0.850	38.50	0.530184	0.469816	1523.94	0.480495	1522.50	1502.94	1528.46	1531.16
8	8	0.825	38.25	0.516195	-0.016195	1531.16	-0.007198	1528.46	1519.90	1525.70	1530.91
9	10	0.875	39.25	0.556225	-0.041232	1530.91	-0.036468	1531.98	1491.68	1523.01	1530.45
10	11	0.850	39.00	0.524816	0.754308	1530.45	0.761990	1523.01	1513.20	1540.30	1542.34
11	12	0.825	38.75	0.593509	-0.090392	1542.34	-0.070030	1540.30	1476.58	1540.10	1541.20
12	13	0.800	38.50	0.499128	0.500872	1541.20	0.500395	1540.10	1541.81	1549.62	1548.93
13	14	0.775	38.25	0.569063	0.824352	1548.93	0.850693	1549.62	1500.63	1551.91	1552.55
14	15	0.750	38.00	0.662249	-0.162249	1552.55	-0.133822	1551.91	1435.58	1557.36	1550.99
15	16	0.725	37.75	0.602320	-0.102320	1550.99	-0.080700	1557.36	1478.87	1550.34	1549.88

4.4.3 Seasonal Elo Calculation

The calculation proceeded as follows:

Initialization. All players and teams began with an Elo rating of 1500. Player-specific parameters were initialised as $k_{Ai} = 32$ (bounded between 24–40) and $q_{Ai} = 1.0$ (bounded between 0.5–1.0)[36].

Instance assignment. Each team's matches were sequentially enumerated as team instances. Each player's appearances were enumerated as player instances, enabling tracking of match participation history. By implementing this method, previous iterations of the variables can be called while looping through the rows, enabling the usage of historical values while calculating their new counterparts.

Dynamic parameter updates.

- k_{Ai} increased by $+0.5$ if the player missed one or more matches (uncertainty due to inactivity)[36].
- k_{Ai} decreased by -0.25 if the player played consecutively (greater rating stability)[36].
- q_{Ai} increased by $+0.025$ when absent and decreased by -0.025 when present, bounded within [0.5, 1][36].

The chance of success. For each player:

$$E_{Ai} = \frac{1}{1 + 10^{(R_B - R_{Ai})/400}}$$

where R_B is the opponent team's average Elo. EA_i represents the expected score (or expected result) of player i in a given match, based on the Elo framework. It is the probabilistic outcome of the match from the perspective of player i, calculated by comparing the rating of the player (or the team the player belongs to) against the opponent's rating[36].

Adjustment factor.

$$C_{Ai} = \begin{cases} w \cdot (S_{Ai} - E_{Ai}) \cdot \frac{M_{Ai}}{M_{\max}}, & \text{if } D_{Ai} = 0 \\ w \cdot (S_{Ai} - E_{Ai}) \cdot |D_{Ai}|^{1/3}, & \text{otherwise} \end{cases}$$

where D_{Ai} is the goal difference, M_{Ai} is minutes played, $M_{\max}$ is maximum minutes, and S_{Ai} is the actual match result (1 =win, 0.5 =draw, 0 =loss). The symbol "w" represents match importance but for the scope of this project, it was set to "1" for all calculations[36].

New ratings. Player ratings were updated as:

$$R_{Ai}^{new} = R_{Ai} + k_{Ai} \cdot C_{Ai} \cdot (1 - E_{Ai}) \cdot \frac{M_{Ai}}{M_{\max}} \cdot q_{Ai}$$

[36]

Table 4.2: Summary of variables used in the Seasonal Elo calculation via Wolf et al.[36]

Symbol	**Description**	**Formula**
R_{Ai}	Elo rating of player A_i before the match	Initialised at 1500; updated after each match
R_A	Team Elo rating before the match	Minutes-weighted average of all R_{Ai} values
R_B	Opponent team Elo rating	Minutes-weighted average of opponent's players
E_{Ai}	The expected chance of success of player A_i	$E_{Ai} = \frac{1}{1 + 10^{(R_B - R_{Ai})/400}}$
S_{Ai}	Actual score outcome for player A_i	$S_{Ai} = 1$ (win), 0.5 (draw), 0 (loss)
D_{Ai}	Goal difference from perspective of player A_i's team	$D_{Ai} = G_A - G_B$ (home) or $G_B - G_A$ (away)
M_{Ai}	Minutes played by player A_i	Observed match data ($M_{Ai} \leq M_{\max} = 90$)
k_{Ai}	Player-specific K-factor (update strength)	Updated per match: $+0.5$ if missed, -0.25 if played; bounded $[24, 40]$
q_{Ai}	Player contribution scaling factor	Updated per match: $+0.025$ if missed, -0.025 if played; bounded $[0.5, 1]$
C_{Ai}	Player adjustment factor	$\begin{cases} (S_{Ai} - E_{Ai}) \cdot \lvert D_{Ai} \rvert^{1/3}, & \text{if } D_{Ai} \neq 0 \\ (S_{Ai} - E_{Ai}) \cdot \frac{M_{Ai}}{M_{max}}, & \text{if } D_{Ai} = 0 \end{cases}$
R_{Ai}^{new}	Updated player Elo after match	$R_{Ai}^{new} = R_{Ai} + k_{Ai} \cdot C_{Ai} \cdot (1 - E_{Ai}) \cdot \frac{M_{Ai}}{M_{max}} \cdot q_{Ai}$
R_A^{new}	Updated team Elo after match	$R_A^{new} = \frac{\sum (R_{Ai}^{new} \cdot M_{Ai})}{\sum M_{Ai}}$

Team ratings. The new team rating R_A^{new} was computed as the minutes-weighted average of updated player ratings. A corresponding aggregate adjustment factor C_A was also calculated.

This process was applied separately for each season (2015/16–2023/24), producing full histories of player-level and team-level Seasonal Elo[36].

Scaling. Historically, the probability of the home team winning a football match has been shown to be significantly higher than that of the away team [23, 33]. To account for this phenomenon, a home advantage term is typically incorporated into Elo-based predictive models. In the classical Elo system, this advantage is

often operationalised as an additional rating of approximately 100 Elo points[15] awarded to the home team prior to each match.

In this project, however, the Elo ratings were recalibrated at the beginning of each season, meaning that the resulting values are constrained within narrower ranges than those produced by long-term historical Elo ratings such as ClubELO. Applying a fixed value of 100 Elo points in this context would therefore distort the relative weighting between home and away performances. To address this, the home advantage was scaled proportionally by comparing the maximum and minimum observed values in the seasonal Elo ratings against those of the historical ClubELO ratings. This ensured that the home advantage remained meaningful and consistent within the recalibrated seasonal framework, while still being anchored to the empirical evidence established in prior studies.

4.4.4 Rationale

The dual inclusion of ClubELO and Seasonal Elo enabled the project to leverage both external long-term team ratings (robust to noise, globally comparable) and season-specific, player-sensitive ratings (responsive to form, minutes, and individual contributions). This hybrid approach provided a richer explanatory basis for match outcome modelling than either dataset alone.

4.5 Feature Aggregation

Raw and engineered features alone are not sufficient for pre-match prediction, since many statistics (e.g., shots, possession, or expected goals) are only observable once a match has been played. To address this, aggregation methods were applied to historical data in order to estimate team or player form prior to each fixture. These aggregated features provided predictive context by capturing trends, smoothing fluctuations, and extrapolating likely values for upcoming matches. Two main approaches were explored: rolling aggregation and aggregation using machine learning.

4.5.1 Rolling Aggregation

Rolling aggregation methods summarise a team's or player's recent history over a defined time window, providing estimates of pre-match statistics based on past performances. These aggregations were either done per player or per team, depending on the type of dataset.

Several techniques were implemented and tested across a range of window sizes (e.g., 4 to 30), with performance measured in terms of prediction error. The following rolling methods were compared:

- **Rolling mean:** Averages feature values over the past n matches, offering a simple baseline for recent performance.
- **Rolling linear regression:** Fits a linear trend across the last n matches, capturing upward or downward form rather than just central tendency.
- **Exponential moving average (EMA):** Assigns exponentially greater weight to recent matches, allowing rapid adaptation to form changes.
- **Weighted moving average (WMA):** Uses custom-defined weights to prioritise recent matches while still retaining earlier history.
- **Rolling median:** Provides a robust alternative to the mean, less sensitive to extreme outliers (e.g., unusually high shot counts or disciplinary events).

Error rates for each aggregation method were computed using MAE, and the optimal method for each feature was identified by locating the knee point of the error curve. Results varied depending on the type of statistic, but for the majority of features the best-performing approaches were either the rolling mean or the rolling median.

Across features, the error curves consistently resembled a negative exponential shape: they began with a sharp decline, reached a knee point, and then stabilised as the window size increased. This pattern indicates that most features were inherently noisy, and that using larger windows helped to smooth this noise by capturing the long-term average around which the feature values fluctuated.

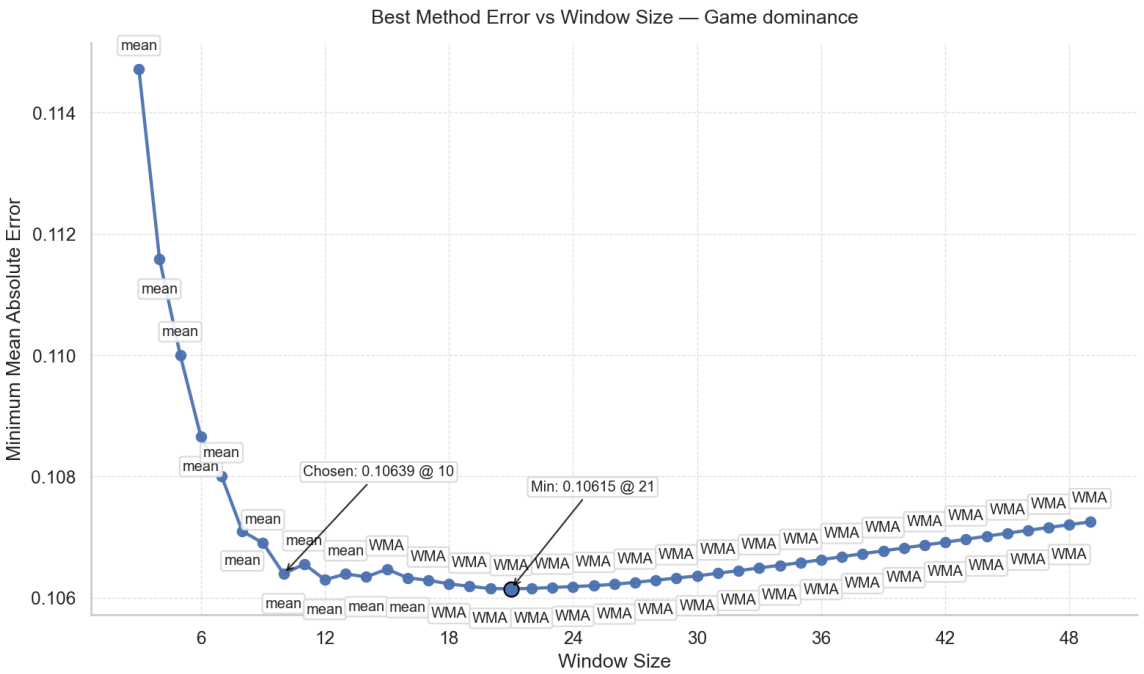


Figure 4.4: Best Method Error vs Window Size – Game Dominance

In cases where the plot extended to very large window sizes, the error rate occasionally began to increase again. This suggests that over-smoothing can occur when the aggregation window becomes too wide, as it dilutes recent information and reduces the model's ability to adapt to short-term variations in team or player form. The pseudocode for which the aggregation functions were implemented can be found in (Appendix B.3

Table 4.3: Example snippet of optimal aggregation method, window size, and mean error for selected features, for Sofascore.

Feature	**Window**	**Method**	**MAE**
Corner kicks	17	mean	2.403070
Ground duels	15	mean	0.051043
Aerial duels	20	mean	0.083467
Blocked per shots on target	15	median	0.678309
Shots on target per total shots	15	mean	0.128647
Hit woodwork per total shots	20	median	0.025993
Shots outside box per total shots	20	mean	0.131789
Big chances	20	mean	1.184795
Fouled in final third	20	mean	1.279678
Dribbles	20	mean	0.407626
Crosses	4	mean	0.194456
Long balls	20	median	0.447829
Game dominance	10	mean	0.106394

4.5.2 Aggregation with Machine Learning

While rolling aggregation provided a simple and interpretable way of estimating pre-match features, its ability to capture non-linear relationships and cross-feature dependencies was limited. To address this, a second approach was implemented using machine learning regressors. In this approach, the window aggregated data were fed in to machine learning algorithms to directly estimate the value of each target feature prior to a match.

Player level Understat roster data was collapsed into its per-match form to simplify the process. The modelling pipeline included the following steps:

1. **Feature set construction.** The input matrix combined three categories of explanatory variables:

   - **Window Aggregated Sofascore/Understat statistics:** Rolling means or medians of features such as shots, passes, tackles, or possession-based measures.

- **Elo ratings:** Both ClubELO ratings and the custom seasonal Elo ratings developed in this project, merged into match records and adjusted by home/away designation.
- **FIFA attributes:** Player-level ratings aggregated to the team level, providing a stable, expert-driven measure of player quality, inverted depending on whether a team played home or away.

2. **Model training.** Three supervised regression algorithms were employed:

   - **Random Forest Regressor (RF):** Trained with 300 trees to capture non-linear feature interactions and provide interpretable feature importance values.
   - **Extreme Gradient Boosting (XGBoost):** A gradient-boosted ensemble tuned with depth, learning rate, and subsampling controls, effective at capturing complex dependencies.
   - **Neural Networks (NN):** Multi-layer feed-forward networks with ReLU activations and dropout regularisation, trained with early stopping to avoid overfitting and using scaled inputs.

   Time series splits were used to preserve chronological order and prevent data leakage. For each fold, performance was evaluated with MAE and R^2.

3. **Feature importance and model selection.** For Random Forests, the five most important predictors for each target feature were extracted and compared across folds to identify consistently influential variables. To balance predictive accuracy with interpretability, two configurations were tested:

   - **Full models:** Trained on the entire feature set (Sofascore/Understat, Elo, FIFA).
   - **Reduced models:** Trained only on the top predictors identified by importance analysis.

   For each target feature, the model with the best performers was retained as the final estimator.

4. **Target features.** A set of match-level statistics were modeled for each dataset, Sofascore and Understat. For discrete variables such as fouls or yellow cards, final predictions were rounded to integers.

5. **Aggregation output.** The trained models were applied across the full dataset to produce new columns of predicted, pre-match feature values (e.g., `fouls_agg`, `corners_agg`, `game_dominance_agg`). These outputs formed the machine-learning aggregated features used in the downstream match outcome classification models.

Machine-learning pipeline pseudocode can be found in (Appendix B.4)

Feature	Rolling Method	Modeling Method	Rolling MAE	Modeling MAE	Improvement
Corner kicks	mean	Neural Network (small)	2.403	2.130	11.3%
Fouls	mean	Random Forest (full)	3.701	1.528	58.7%
Shots off target	mean	Random Forest (full)	2.086	0.880	57.8%
Throw-ins	mean	Random Forest (full)	5.168	2.250	56.5%
Ground duels	mean	Neural Network (small)	0.051	0.049	3.6%
Aerial duels	mean	Random Forest (full)	0.083	0.038	54.7%
Game dominance	mean	XGBoost (full)	0.106	0.034	68.2%
Shots on target / total shots	mean	Neural Network (small)	0.129	0.124	4.0%
Hit woodwork / total shots	median	Neural Network (full)	0.037	0.026	42.0%
Shots outside box / total shots	mean	Neural Network (small)	0.132	0.127	3.5%
Big chances	mean	XGBoost (full)	1.185	0.647	45.4%
Fouled in final third	mean	XGBoost (full)	1.280	0.795	37.9%
Tackles won	mean	Neural Network (full)	0.332	0.328	1.1%
Dribbles	mean	Neural Network (small)	0.408	0.400	1.8%
Crosses	mean	Random Forest (full)	0.194	0.194	0.4%
Long balls	median	XGBoost (full)	0.448	0.360	19.6%
Yellow cards	mean	Random Forest (full)	0.981	0.373	62.0%
Offsides	median	XGBoost (full)	1.159	0.778	32.8%
Goalkeeper saves	mean	Random Forest (full)	1.585	0.670	57.7%

Table 4.4: Example Snippet of Comparison of Rolling vs. ML-Based Aggregation Methods(small:top-n features, full:all features)

4.6 Football-Data.co.uk (Betting Odds)

To benchmark model outputs against market expectations and to create a bookmaker-derived baseline, match odds were sourced from *Football-Data.co.uk* [14] for the English Premier League seasons overlapping this study. The aim was to extract pre-match probabilities for home win, draw, and away win, and align them to the unified match index used elsewhere in the thesis.

4.6.1 Data Collection

CSV files for the EPL were downloaded across multiple seasons and concatenated into a single DataFrame. The dataset was de-duplicated, and only columns relevant to full-time results and odds were retained. Ancillary fields (e.g., referee, first-half statistics) were removed to reduce noise.

4.6.2 Data Cleaning

Odds to implied probabilities. Pinnacle/PS odds (`PSCH`, `PSCD`, `PSCA`) were transformed into naïve implied probabilities via reciprocal conversion[9]:

$$BetChance_{outcome} = \frac{1}{Odds_{outcome}}, \quad outcome \in \{home, draw, away\}$$

No overround normalisation was applied; probabilities were used "as is" to reflect the raw market pricing signal.

4.6.3 Feature Construction and Baseline Evaluation

A bookmaker baseline (`Bet Result`) was assigned by taking the argmax of the three implied probabilities (`home`, `draw`, `away`). This baseline was compared to the labelled outcome to compute:

- overall accuracy,
- per-class accuracies (home win, draw, away win).

This served two purposes: (i) to quantify the inherent difficulty of predicting draws in real markets, and (ii) to provide a market-informed benchmark against which model performance could be contextualised.

4.6.4 Output

The final betting dataset consisted of the following variables:

Table 4.5: Final variables from Football-Data.co.uk betting dataset

Variable	Description
id	Understat match identifier
BetChance_home	Implied probability of home win
BetChance_draw	Implied probability of draw
BetChance_away	Implied probability of away win

4.7 Merging Data

Since the datasets were gathered from different sources, a major challenge was the lack of a universal identifier that could directly link them. Each dataset used its own unique system of primary and secondary keys, making them incompatible for straightforward merging. As a result, the integration process relied primarily on entity names (e.g., team names and player names) as the common fields across datasets.

This approach required extensive standardisation and cleaning of string variables, as naming conventions varied between sources (for example, abbreviations, punctuation, or alternative spellings of club names). In cases where multiple versions of a player or team name existed, mappings and replacement dictionaries were constructed to ensure consistency. Where ambiguities arose (such as duplicated names across different players), contextual attributes like season, match identifier, or team membership were used to disambiguate.

By resolving these differences, a unified dataset was created, allowing match, player, and team statistics to be combined into a single analysis-ready table. The main configuration for this dataset was chosen to follow the **Understat** standard, as it provided the most extensive indexing and key structures compared to the other datasets.

4.7.1 Team Name Matching

Team names represented the simpler part of the matching process within the scope of this project. The number of unique Premier League clubs was relatively small, and therefore inconsistencies across sources could be managed through direct mapping. When mismatched names were encountered, they were manually imputed and aligned with their Understat counterparts.

Source	Original Representation	Unified to Understat Name
Understat	Manchester City	Manchester City
ClubElo	Man City	Manchester City
Football-Data	Man. City	Manchester City
SofaScore	MCI	Manchester City
Understat	Wolverhampton Wanderers	Wolverhampton Wanderers
ClubElo	Wolves	Wolverhampton Wanderers
Football-Data	Wolverhampton	Wolverhampton Wanderers

Table 4.6: Example of team name standardisation across datasets.

4.7.2 Player Name Matching

One of the most extensive and time-consuming parts of the project was the process of player name matching. Unlike team names, which had a limited set of variations, player names across *Understat* and *FIFAIndex*, contained inconsistencies in spelling, special characters, abbreviations, and language accents. This made automated alignment between datasets much more challenging.

Name Cleaning and Standardisation

The first stage involved normalising player names into a comparable format. All strings were lowercased, accents were stripped using the `unidecode` library, and punctuation such as periods or commas was removed. This created a uniform baseline to minimise false mismatches.

```
FUNCTION clean_name(name):
    name = lowercase(name)
    name = remove accents with unidecode
    name = strip punctuation and whitespace
    RETURN name
```

This cleaning was applied to both datasets, resulting in columns `Fifa_Name` and `Understat_Player_Name`. It was found that matching names with these changes resulted in a 50% success rate.

Fuzzy Matching with RapidFuzz

Once names were standardised, `RapidFuzz` was used to compute string similarity scores between `Understat_Player_Name` and the canonical FIFA names. Each Understat name was assigned its closest FIFA counterpart and given a confidence score. Thresholds were then used to group matches:

- **High-confidence matches** ($\geq 90\%$) $\rightarrow$ merged directly, but was still inspected.
- **Medium-confidence matches** (80–90%) $\rightarrow$ flagged for manual inspection.
- **Low-confidence matches** (¡80%) $\rightarrow$ required manual review or remained unmatched.

```
FOR each Understat_Player_Name:
    best_match, score = fuzzy_match(Fifa_Name list)
    IF score >= 90:
        auto-merge
    ELSE IF score >= 80:
        flag for inspection
    ELSE:
        mark as unmatched
```

Manual Validation

Borderline and low-confidence matches were exported to Excel for manual inspection. These cases often involved players with common surnames (e.g., “Fernandes”), shortened forms (e.g., “A. Smith” vs. “Adam Smith”), or language-specific characters. Manual confirmation ensured accuracy where string-based methods were insufficient.

Unfortunately, within the time constraints of the project, it was not feasible to manually impute all mismatched player names. Therefore, if a player’s name could not be resolved during the manual inspection stage, their entry was imputed using the average FIFA ratings corresponding to their positional role.

Please refer to (Appendix C.1 for the final merged dataset.)

4.8 Exploratory Data Analysis (EDA)

Most of the exploratory work was already carried out in earlier sections when describing each data source, detailing how raw values were cleaned, transformed, and engineered into usable features. Issues of collinearity, redundancy, and relationships between variables have also been addressed during feature engineering.

This section therefore focuses on examining the structure and quality of the final compiled dataset before modelling. The aim is to understand the marginal distributions and scales of key variables, evaluate how the engineered features contributed to a consistent representation of team and player performance, and confirm that the resulting dataset is robust and ready for predictive analysis.

4.8.1 Seasonal Elo Per Position



Figure 4.5: Best Seasonal Elo Ratings Of All Seasons, per Position Played

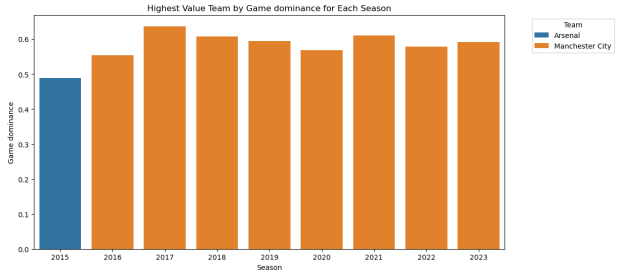
A pitch-based visualization (Figure 4.5) was created to show players' peak seasonal Elo ratings, their most common positions, and the seasons in which these values occurred. The results indicated that top performances were spread across the pitch, with forwards and creative midfielders (e.g., Dele Alli 2016, David Silva 2017, Mohamed Salah 2019) reaching Elo values in the 1590–1640 range, while defenders and goalkeepers such as Aymeric Laporte (2021) and Ederson (2017) also achieved similarly high scores. This demonstrated that peak seasonal Elo was not restricted to attacking roles but evident across all positions. [28, 13]

4.8.2 Game Dominance Across Seasons

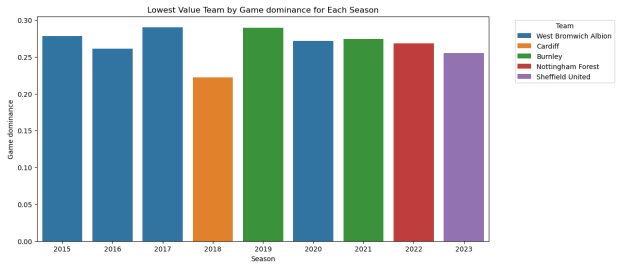
Game Dominance was defined as:

$$\text{Game Dominance} = \text{Ball Possession} \times \text{Pass Accuracy}$$

capturing both territorial control and technical efficiency. Higher values typically characterise title contenders, while lower values align with relegation struggles.



(a) Highest Game Dominance values across seasons.



(b) Lowest Game Dominance values across seasons.

Figure 4.6: Comparison of teams with highest and lowest Game Dominance across Premier League seasons.

High Dominance Teams. From 2016 onwards, Manchester City consistently led this metric under Guardiola, reflecting their possession-oriented style and coinciding with five Premier League titles. Arsenal appeared briefly in 2015 under Wenger's similar philosophy.

Low Dominance Teams. Lowest dominance values almost always coincided with relegation battles: West Brom (2015–17, 2020), Cardiff (2018), Burnley (2019, 2021), Nottingham Forest (2022), and Sheffield United (2023). Most were relegated in those seasons or narrowly survived.

Summary. Across nine seasons, high dominance is consistently linked to champions, while low dominance identifies relegation-threatened teams, underscoring the metric's explanatory and predictive value.

Chapter 5
Methodology

5.1 Summary of Pre-processing

The data collection process drew from five primary sources: Understat, Sofascore, FIFAIndex, ClubELO, and Football-Data.co.uk. Each source provided complementary information, ranging from in-game statistics and advanced performance metrics to expert-driven player ratings and bookmaker odds.

Scraping and Collection

Data were retrieved using a combination of open-source tools (`ScraperFC`), browser automation with Selenium Stealth, and direct queries to public APIs where available (e.g., Sofascore). The scraping pipelines included retry mechanisms and checkpointing to ensure resilience against connection failures.

Data Cleaning

Substantial preprocessing was required to harmonise formats:

- Conversion of percentage-based statistics into numeric floats,
- Normalisation of inconsistent labels (e.g., duel formats, passing stats),
- Regression-based imputation for missing values, with features dropped if NaNs exceeded thresholds,
- Deduplication and standardisation of season/version identifiers.

Feature Engineering

Several new metrics were constructed to improve explanatory power:

- Ratio-based efficiency metrics (e.g., shots on target per total shots),
- Composite indicators such as *Game Dominance* (possession $\times$ pass accuracy),
- Aggregated FIFA positional/skill-group ratings and adjusted attributes,
- Player-level Seasonal Elo ratings extended from Elo frameworks to capture individual contributions, minutes played, and seasonal resets.

Indexing and Standardisation

Each dataset relied on different identifiers, so team names, player names, and match IDs were systematically aligned to the Understat standard. Player names required fuzzy matching (`RapidFuzz`) and manual inspection, with unresolved mismatches imputed via positional FIFA averages.

Prepared for Merging

At this stage, each dataset was cleaned, feature-engineered, and indexed consistently. This created the foundation for merging into a unified analysis-ready dataset where match-, team-, and player-level information could be combined. Football-Data.co.uk was spared to be used to evaluate model performance, as it is conisted of betting odds.

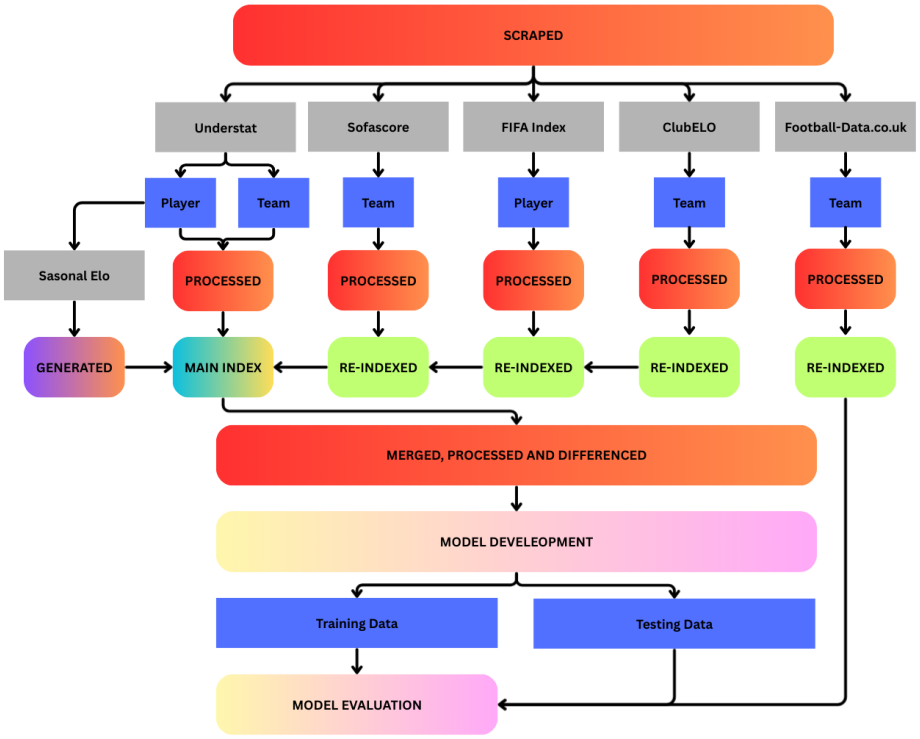


Figure 5.1: Workflow [5]

The final dataset was structured at the match level, with each row representing a single fixture and containing features for both the home and away teams. To standardise the representation, the dataset was pivoted by subtracting the away team's features from the corresponding home team's features. As a result, positive values indicate home-team dominance in that feature, while negative values indicate away-team dominance.

$$\text{feature}_{\text{home}} - \text{feature}_{\text{away}} = \text{feature}$$

5.2 Modelling

5.2.1 Problem Setup and Splits

The task is multiclass outcome prediction (**Home Win / Draw / Away Loss**). After merging all sources into a single chronologically indexed table, matches were sorted by date and split by time.

5.2.2 Feature Families

All feature sets were augmented with FIFA (team-aggregated player attributes) and Elo (ClubELO + Seasonal Elo).

Original (post-match stats): DEEP, PPDA, xG, key passes, game dominance, cards, duels, etc.

Window-aggregated (rolling): Per-team rolling means/medians (e.g., `Fouls_mean`, `xG_median`, `PPDA_median`) to be compared against aggregation.

Model-aggregated: Features which were aggregated using machine learning.

5.2.3 Baselines and Classifiers

Three classifiers were trained with consistent label encoding (W/D/L) and class-imbalance handling (balanced class weights and/or loss):

- **Random Forest (scikit-learn)** for strong tabular baselines and permutation importance.
- **XGBoost** with multiclass objective.
- **Neural Networks (TensorFlow/Keras)** with scaling, early stopping, and class weighting/focal loss to better capture the Draw class.

5.2.4 Evaluation Tracks

Model and Training Feature Families

A key challenge is that many raw features (e.g., shots, possession, xG) are only available *after* a match has been played, since these metrics generate as the game is played, making them unusable for genuine pre-match prediction. Nevertheless, these post-match values remain highly valuable within the modeling framework. They serve as a important benchmark, since they reflect the actual realised behaviour of teams on the pitch.

Original features represent data that has actually happened during the match, so it has the ultimate ability to capture how those features reflect on the outcome of the matches.

By comparing model outputs trained or tested on these realised values against aggregated or predicted proxies, we gain a clear understanding of:

- how well aggregated features approximate true in-match performance,
- the extent of error introduced by using pre-match estimators instead of realised outcomes, and
- whether the models are learning patterns that remain robust when exposed to noisier, proxy-based representations.

In this way, post-match statistics are not used as predictive inputs, but rather as a diagnostic tool to validate the effectiveness of feature engineering and aggregation strategies.

Like-for-Like Training

Separate models were trained and tested on each family of features to establish fair comparisons:

$$\begin{aligned} \text{Train: Original} &\rightarrow \text{Test: Original} \\ \text{Train: Window Aggregated} &\rightarrow \text{Test: Window Aggregated} \\ \text{Train: ML Aggregated} &\rightarrow \text{Test: ML Aggregated} \end{aligned}$$

In addition, proxy-shift experiments were conducted where models trained on post-match *Original* features were evaluated on aggregated proxies that were mapped back to the same feature names:

$$\begin{aligned} \text{Train: Original} &\rightarrow \text{Test: ML Aggregated (mapped)} \\ \text{Train: Original} &\rightarrow \text{Test: Window Aggregated (mapped)} \end{aligned}$$

This setup allowed direct assessment of both the robustness of models to covariate shift and the relative predictive value of each feature construction approach.

Purpose: determine which representation performs best when the classifier is trained on the features it will actually see at inference.

5.2.5 Feature Selection for Parsimony

The final dataset for all three feature families contained 57 features across 3,440 observations. While ensemble methods such as Random Forests are generally robust to moderately high dimensionality, a feature space of this size brings the definite case of overfitting.

To address this, Random Forests were first employed to compute permutation importance, evaluated using macro-F1 on the chronological test set. For each feature family, the ten most important predictors were retained. Reduced models (Random Forest, XGBoost, and Neural Networks) were subsequently re-trained using only these Top-10 subsets. This procedure enabled the removal of weak or collinear features, ensured that feature selection was directly aligned with predictive performance, and resulted in more parsimonious models without compromising accuracy.

5.2.6 Pre-result Realization

It was observed that training and testing on aggregated features (AGG $\rightarrow$ AGG) produced results most comparable to the baseline case of training and testing on realised post-match features (Original $\rightarrow$ Original). As expected, window-based proxies performed worse, since machine learning (ML) aggregation generated features that were closer in distribution to the original statistics. However, the fact that AGG $\rightarrow$ AGG outperformed AGG $\rightarrow$ Original carries several important implications.

1. **Covariate shift.** Realised (post-match) features and pre-match aggregates follow different distributions. Aggregates typically have lower variance and different scaling. A model trained on realised data becomes miscalibrated when tested on aggregates, whereas training directly on AGG ensures the model learns boundaries that reflect the aggregated distribution.

2. **Latency alignment.** Realised statistics are only known after a match, while AGG features are pre-match estimates. Training on AGG therefore aligns the training process with the information actually available at inference, producing more realistic decision rules.

3. **Noise reduction.** Aggregation reduces random match-to-match fluctuations, making patterns more stable and predictive. Models trained directly on AGG learn from this cleaner signal, while models trained on realised features may overfit to noise that will not be present in pre-match estimates.

4. **Class balance sensitivity.** The Draw class is relatively rare and highly sensitive to shifts in feature distributions. When class weights or focal loss are tuned on AGG features, recall and precision for draws are better preserved compared to applying weights learned from realised features.

Takeaway: In time-series tabular machine learning, models should be trained using the same feature construction that will be available at inference. This minimises covariate shift, ensures realistic alignment, and improves robustness to noise and class imbalance.

5.2.7 Neural-Network Tuning (AGG→AGG)

A small random search with time-series CV (no leakage) explored:

$$\{\text{depth/width, dropout, } \ell_2, \text{ learning rate, batch size, draw_mult, loss type}\},$$

where loss was cross-entropy vs. focal loss (per-class α, γ). The best configuration was refit on the full training block with an internal chronological validation slice for early stopping, then evaluated on the held-out test block.

Chapter 6
Results and Thoughts

6.1 Master Performance Summary

Table 6.1: Test performance across model families and train→test regimes. "Features" indicates the feature set used for training (and testing, unless noted). For proxy experiments, models trained on ORIGINAL are evaluated on AGG/WINDOW proxies. Metrics are on the held-out test set.

Model	Train → Test	Features	Accuracy	Macro-F1	Draw F1
Full features (no feature selection)					
RF	ORIGINAL → ORIGINAL	Full	0.6827	0.5655	0.18
RF	ORIGINAL → AGG (proxy)	Full	0.5921	0.5033	0.21
RF	ORIGINAL → WINDOW (proxy)	Full	0.5336	0.4757	0.24
RF	AGG → AGG	Full	0.6257	0.5105	0.15
RF	WINDOW → WINDOW	Full	0.5760	0.4387	0.05
Top-10 features (per family-specific permutation importance)					
RF	ORIGINAL → ORIGINAL	Top-10	0.6798	0.6097	0.33
RF	AGG → AGG	Top-10	0.6418	0.5578	0.25
RF	WINDOW → WINDOW	Top-10	0.5629	0.4725	0.17
RF	ORIGINAL → AGG (proxy)	Top-10	0.6184	0.5343	0.23
RF	ORIGINAL → WINDOW (proxy)	Top-10	0.5614	0.4636	0.15
XGB	ORIGINAL → ORIGINAL	Top-10	0.6798	0.6150	0.33
XGB	AGG → AGG	Top-10	0.6433	0.5605	0.25
XGB	WINDOW → WINDOW	Top-10	0.5249	0.4428	0.16
XGB	ORIGINAL → AGG (proxy)	Top-10	0.6082	0.5183	0.19
XGB	ORIGINAL → WINDOW (proxy)	Top-10	0.5234	0.4358	0.13
NN	**AGG → AGG**	**Top-10**	**0.6594**	**0.6024**	**0.34**
NN	WINDOW → WINDOW	Top-10	0.5292	0.4767	0.22
NN	ORIGINAL → AGG (proxy)	Top-10	0.6184	0.5772	0.35
NN	ORIGINAL → WINDOW (proxy)	Top-10	0.5439	0.4737	0.20

6.2 Neural Network Hyperparameter Tuning (AGG → AGG)

Table 6.2: Neural Network tuning on AGG→AGG (Top-10 features). Metrics are on the held-out test set.

Setup	Acc.	Macro-F1	Draw F1	n_{hidden}	units	dropout	ℓ_2	lr	batch	loss	γ/draw_mult
Baseline tuned	0.6579	0.5880	0.29	3	32	0.4	0.0005	0.005	64	CE	– / –
Draw-weight search	**0.6564**	**0.6118**	**0.37**	**1**	**128**	**0.3**	**0.0005**	**0.002**	**64**	**CE**	**– / 1.0**
Focal vs CE + draw_mult	0.6316	0.5859	0.33	2	96	0.3	0.0010	0.005	32	Focal	2.5 / 1.0

6.3 Discussion of Model Performance

Performance was most stable when models were trained and tested within the same representation, with AGG→AGG consistently outperforming WINDOW→WINDOW. Across models, AGG→AGG achieved higher accuracy and macro-F1, and feature selection further improved performance by reducing overfitting. While draw prediction remained the weakest class overall, the Neural Network on AGG→AGG with Top-10 features gave the best balance, achieving accuracy of 0.6594, macro-F1 of 0.6024, and draw-F1 of 0.34. This shows that aggregation guided by model importance preserves more predictive signal than simple rolling windows and supports more balanced performance across classes.

Hyperparameter tuning on the AGG→AGG setup provided additional insights. A baseline tuned NN improved calibration but offered limited gains for draws. Incorporating draw-weighting yielded the strongest result, raising draw-F1 to 0.37 and macro-F1 to 0.6118 while maintaining overall accuracy.

6.4 Discussion of Feature Importances

In both the ORIGINAL and AGG→AGG settings, the most influential features align with measures of chance creation, control, and team strength, supplemented by engineered variables that capture more nuanced aspects of play. For the ORIGINAL dataset, model importance highlighted *xG*, *Shots on target per total shots*, and *xGBuildup* as central, reflecting the value of expected goals and build-up contributions in discriminating match outcomes. Alongside these, engineered or curated features such as *Overall* player rating, *Agility_adj*, and *Stamina_adj* surfaced, indicating that physical and technical player attributes provide complementary predictive signal beyond purely match-derived statistics. The presence of *Goalkeeper saves* and *Blocked per shots on target* suggests that defensive resilience is also materially informative.

In the AGG dataset, aggregation emphasised longer-term structural signals. *xG_agg* and *xGBuildup_agg* remained central, but engineered control metrics such as *Game dominance_agg* and *DEEP_agg* rose in importance, showing that aggregated possession and territory measures contribute substantially when smoothed over windows. Team strength proxies like *Seasonal_Elo* and *ClubELO* further stabilised predictive power by capturing opponent-adjusted form across the season. Engineered event counts (*Throw-ins_agg*, *Shots off target_agg*) and retained physical attributes (*Stamina_adj*) indicate that small technical details, when consistently aggregated, can add discriminative information. Overall, the feature importances confirm that engineered and generated constructs designed in this study—particularly *Game dominance*, **Seasonal Elo**, and adjusted player ratings—play a decisive role in complementing canonical performance metrics such as xG and DEEP.

Table 6.3: Top-10 most important features for ORIGINAL and AGG→AGG models.

ORIGINAL (Top-10)	**AGG (Top-10)**
xG	xG_agg
Shots on target per total shots	xGBuildup_agg
xGBuildup	Game dominance_agg
Overall	DEEP_agg
ClubELO	Seasonal_Elo
Goalkeeper saves	Throw-ins_agg
Blocked per shots on target	key_passes_agg
Agility_adj	Shots off target_agg
key_passes	ClubELO
Stamina_adj	Stamina_adj

6.5 Comparison to Betting Odds

To contextualise model performance, predictions were compared against bookmaker implied outcomes. On the full dataset of 3,420 matches, betting odds achieved an accuracy of 0.554. Restricting to the 684 test fixtures used for model evaluation, betting odds attained 0.577 accuracy, with no success in correctly predicting draws.

By contrast, the draw-weighted neural network (AGG→AGG, Top-10 features) reached 0.632 accuracy on the same test set. Most notably, it achieved a draw recall of 0.333, compared to 0.0 for the betting baseline, indicating that the class-aware objective improved minority-class recognition without compromising overall calibration. In balanced betting markets (home win chance between 0.45–0.55), both approaches produced similar accuracies (≈0.55–0.58), suggesting that the advantage of the NN is most pronounced in cases where bookmakers traditionally struggle—specifically the draw outcome.

Table 6.4: Comparison of betting odds and draw-weighted NN performance on the test set (684 matches).

Method	**Accuracy**	**Macro-F1**	**Draw F1**
Betting odds (baseline)	0.577	0.423	0.000
NN (AGG→AGG, Top-10, draw-weighted)	0.632	0.612	0.333

Chapter 7
Conclusion

Conclusion

This thesis developed and evaluated machine learning models for football match outcome prediction by combining event-based statistics, aggregated features, and player-level attributes with external strength ratings. A full scraping–merging pipeline was implemented using multiple sources including Understat, SofaScore, FIFAIndex, and ClubElo, enabling the construction of enriched datasets across nine Premier League seasons. Experiments showed that training and testing within the same representation was essential for stability, with AGG→AGG outperforming WINDOW-based features. Among the tested families, a neural network trained on Top-10 AGG features with a draw-weighted loss achieved the best overall balance, reaching 0.632 accuracy, 0.612 macro-F1, and 0.333 draw-F1 on the test set. This model not only surpassed ensemble baselines but also outperformed bookmaker odds on the same fixtures, highlighting the effectiveness of compact, importance-guided aggregation and modest class weighting.

Research Contributions

The primary contribution of this work is the demonstration that importance-driven aggregation can yield more robust predictive performance than both raw and rolling representations. By engineering new features such as *Game dominance* and *DEEP*, and by incorporating adjusted FIFA attributes alongside established measures like expected goals and Elo ratings, the study provided evidence that enriched multi-source signals substantially improve prediction of match outcomes. Neural networks proved particularly effective when tuned with class-aware weighting, enhancing recognition of draws, which remain the hardest outcome to model. The benchmarking exercise against bookmaker odds further emphasised the utility of the approach, as the model delivered both higher overall accuracy and markedly better performance on the minority class. These results align with prior research on Elo-based forecasting [15, 31, 36], player evaluation systems [25, 16, 20], and machine learning models for match prediction [3, 9], extending them by introducing novel engineered constructs and validating their impact in outcome prediction.

Challenges and Limitations

Several challenges were encountered over the course of the project. The process of merging entities across data sources was complex, particularly when aligning player names between FIFAIndex and other platforms, requiring manual interventions and increasing the likelihood of residual mismatches. The scarcity of publicly available, comprehensive datasets also constrained the range of features that could be included, while time limitations restricted the breadth of hyperparameter searches and the exploration of more advanced modelling frameworks. Most importantly, class imbalance continued to limit performance, as draw prediction accuracy lagged behind other outcomes despite improvements from weighting. Furthermore, evaluations across different representations demonstrated sensitivity to feature definitions, with cross-proxy testing (e.g., ORIGINAL→WINDOW) leading to performance degradation.

Future Works

Future research could expand this framework in several directions. Incorporating contextual measures such as match importance, league position, or knockout status could capture psychological and tactical effects that influence results but are not reflected in standard metrics. Additional spatio-temporal and tactical features, such as formations, pressing intensity, or adjusted plus-minus ratings, could provide a richer description of team and player behaviour. Improvements in entity resolution would enable more reliable integration of cross-source data without manual intervention, while probabilistic calibration methods could ensure model probabilities are better aligned with observed frequencies. Finally, extending the dataset to other leagues and employing sequential or hierarchical models would strengthen generalisability, creating a more comprehensive and practical tool for both analysts and decision-makers in football.

Bibliography

[1] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viegas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng. Tensorflow: Large-scale machine learning on heterogeneous systems. `https://www.tensorflow.org/`, 2015.

[2] G. Baio and M. Blangiardo. Bayesian hierarchical model for the prediction of football results. *Journal of Applied Statistics*, 37(2):253–264, 2010.

[3] R. Bunker and T. Susnjak. The application of machine learning algorithms for football match result prediction. *Applied Computing and Informatics*, 15(2):93–107, 2019. doi: 10.1016/j.aci.2017.09.005.

[4] M. Caley. Expected goals (xg) models in football. `https://statsbomb.com/articles/soccer/expected-goals-explained/`, 2015. Accessed: 2025-08-30.

[5] Canva. Canva – graphic design platform. `https://www.canva.com/`. Accessed: 2025-09-01.

[6] M. Cattelan. Models for paired comparison data: A review with emphasis on dependent data. *Statistical Science*, 27(3):412–433, 2012.

[7] T. Chen and C. Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. ACM, 2016. doi: 10.1145/2939672.2939785.

[8] ClubElo. Club elo ratings. `http://clubelo.com/`. Accessed: 2025-08-30.

[9] A. C. Constantinou, N. E. Fenton, and M. Neil. Bayesian networks for the prediction and analysis of football outcomes. *Knowledge-Based Systems*, 36:322–339, 2012. doi: 10.1016/j.knosys.2012.07.008.

[10] T. Decroos, L. Bransen, J. Van Haaren, and J. Davis. Actions speak louder than goals: Valuing player actions in soccer. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1851–1861. ACM, 2019. doi: 10.1145/3292500.3330758.

[11] M. J. Dixon and S. G. Coles. Modelling association football scores and inefficiencies in the football betting market. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 46(2):265–280, 1997.

[12] FIFAIndex. Fifa player and team database. `https://www.fifaindex.com/`. Accessed: 2025-08-30.

[13] FifaRosters. Fifarosters – fifa team and player database. `https://fifarosters.com/`. Accessed: 2025-09-01.

[14] Football-Data.co.uk. Historical football betting odds and results. `https://www.football-data.co.uk/`. Accessed: 2025-08-30.

[15] L. M. Hvattum and H. Arntzen. Using elo ratings for match result prediction in association football. *International Journal of Forecasting*, 26(3):460–470, 2010.

[16] T. Kharrat, A. Guitart, J. Perarnau, and L. Pappalardo. The value of players in soccer: A comprehensive player evaluation approach using plus-minus ratings. *PLOS ONE*, 16(8):e0255316, 2021. doi: 10.1371/journal.pone.0255316.

[17] Z. Király. The elo rating system and its connection to the Bradley–Terry model. *Mathematical Social Sciences*, 79:11–23, 2015.

[18] M. J. Maher. Modelling association football scores. *Statistica Neerlandica*, 36(3):109–118, 1982.

[19] F. Matano, H. Qin, B. Schölkopf, A. Habrard, and L. Pappalardo. Augmenting adjusted plus-minus in soccer with fifa ratings. *Machine Learning*, 111(10):3737–3763, 2022.

[20] L. Pappalardo, P. Cintia, A. Rossi, E. Massucco, P. Ferragina, D. Pedreschi, and F. Giannotti. Playerank: Data-driven performance evaluation and player ranking in soccer via a machine learning approach. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 10(5):1–27, 2019.

[21] L. Pappalardo, E. Massucco, P. Cintia, P. Ferragina, D. Pedreschi, and F. Giannotti. A public data set of spatio-temporal match events in soccer competitions. *Applied Sciences*, 11(5):2378, 2021. doi: 10.3390/app11052378.

[22] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay. Scikit-learn: Machine learning in python, 2011. URL `https://scikit-learn.org/`.

[23] R. Pollard. Home advantage in soccer: A retrospective analysis. *Journal of Sports Sciences*, 4(3):237–248, 1986. doi: 10.1080/02640418608732122.

[24] H. Rue and Ø. Salvesen. Prediction and retrospective analysis of soccer matches in a league. *Scandinavian Journal of Statistics*, 27(2):463–480, 2000. doi: 10.1111/1467-9469.00198.

[25] S. Schultze and C.-M. Wellbrock. A weighted plus/minus metric for individual soccer player performance. *Journal of Sports Analytics*, 4(2):121–131, 2018.

[26] ScraperFC. Scraperfc: Football (soccer) data scraper for python. `https://pypi.org/project/scraperfc/`. Accessed: 2025-08-31.

[27] Selenium Project. Selenium webdriver. `https://www.selenium.dev/documentation/webdriver/`, 2025. Accessed: 2025-08-31.

[28] ShareMyTactics. Sharemytactics – football lineup builder. `https://sharemytactics.com/`. Accessed: 2025-09-01.

[29] Sofascore. Sofascore live football statistics and api. `https://www.sofascore.com/`. Accessed: 2025-08-30.

[30] L. Szczeciński. On the evaluation of the fifa rating system for national teams. *Statistical Modelling*, 19(5): 445–465, 2019.

[31] M. Tavana, F. Azizi, F. Azizi, and M. Behzadian. A football player rating system based on the elo rating method. *International Journal of Applied Decision Sciences*, 13(4):420–444, 2020. doi: 10.1504/IJADS.2020.110749.

[32] C. Trainor. Measuring defensive pressure. `https://statsbomb.com/articles/soccer/measuring-defensive-pressure/`, 2013. Accessed: 2025-08-31.

[33] D. Trombley. Home advantage in soccer: Variations in strength, style, and significance. *Journal of Quantitative Analysis in Sports*, 12(1):27–38, 2016. doi: 10.1515/jqas-2015-0027.

[34] Understat. Understat football analytics platform. `https://understat.com/`. Accessed: 2025-08-30.

[35] J. Van Haaren and G. Van den Broeck. Relational learning for football-related predictions. In *Proceedings of the 21st European Conference on Artificial Intelligence*, pages 747–752. IOS Press, 2012.

[36] S. Wolf, M. Schmitt, and B. Schuller. A football player rating system. *Journal of Sports Analytics*, 6(4): 1–15, 10 2020. doi: 10.3233/JSA-200411.

[37] D. Yeung, R. Bunker, and K. Fujii. Interpretable football match result prediction using team formation and fifa ratings. In *Proceedings of the 2023 IEEE International Conference on Data Mining (ICDM)*, pages 940–945. IEEE, 2023.

Chapter A
Appendix

The source code for this project is available on the University of Birmingham's Git server at the following link: https://git.cs.bham.ac.uk/projects-2024-25/yxy481/-/tree/main/statistical-analysis-and-modeling-of-football-data?ref_type=heads

The code and corresponding results are provided in the Jupyter Notebook, which contains a total of seven files: thirty three coding files and csv files.

The coding files cover various aspects of the project, including scraping, pre-processing and feature engineering, EDA and modeling.

Python version 3.12.4 was used for the project.

Keep in mind that because of the nature of the file branches, some code where it is importing csv's may give errors, please refer to the original file name that is trying to be imported and rename the path to the file accordingly.

DO NOT RUN THE SCRAPING NOTEBOOKS, unless the csv files are expected to be re-scraped. If that is the case, please keep in mind that it might take up to a day for the scraping algorithms to complete. Do pip install scraperfc if that is desired.

A.1 File branches:

The files must be run in the order per section, if running the codes are desired.

A.1.1 Understat

- **Data:** csv files
- **understat_scrape.ipynb:** Scraping algorithms
- **understat_cleaning.ipynb:** Cleaning and correlation analysis, some EDA
- **understat_eda.ipynb:** Exploratory Data Analysis
- **understat_model.ipynb:** A test model to sanity check data, analyze feature importances etc.

A.1.2 Sofascore

- **Data:** csv files
- **Scraping.ipynb:** Scraping algorithms
- **Cleaning.ipynb:** Cleaning and correlation analysis, some EDA
- **sofascore_eda.ipynb:** Exploratory Data Analysis
- **Modeling.ipynb:** A test model to sanity check data, analyze feature importances etc.
- **merging_with_understat.ipynb:** Name matching and creating common keys to be merged with understat indexes.

A.1.3 Football-data-website

- **Data:** csv files
- **footbaldatacom.ipynb:** importing csv's, EDA, correlation analysis, feature engineering and name matching/merging with the understat keys

A.1.4 Fifa

- **Data:** csv files
- **fifa_scrape.ipynb:** Scraping algorithms
- **fifa_cleaning.ipynb:** Cleaning and some EDA
- **fifa_name_matching.ipynb:** The main name matching algorithm
- **fifa_correlation_analysis.ipynb:**correlation analysis
- **scraper.log:** Log files for the scraping algorithm, can be referred to if the nature of the scraping algorithms are misunderstood by the supervisor.

A.1.5 ClubELO

- **Data:** csv files
- **clubelo_scraping.ipynb:** Scraping algorithms
- **player_elo_calculation.ipynb:** Manufacturing of the seasonal elo, algorithms and merging with clubelo

A.1.6 Aggregation

- **Data:** csv files
- **understat_roster_window_agg.ipynb:** Window aggregation algorithms for understat roster dataset (player level)
- **understat_model_window_agg.ipynb:** Machine learning aggregation algorithms for understat roster dataset (converted to team level)
- **understat_team_window_agg.ipynb:** Window aggregation algorithms for understat match dataset (team level)
- **understat_model_window_agg.ipynb:** Machine learning aggregation algorithms for understat match dataset (team level)
- **sofa_team_window_agg.ipynb:** Window aggregation algorithms for sofascore match dataset (team level)
- **sofa_model_window_agg.ipynb:** Machine learning aggregation algorithms for sofascore match dataset (team level)

A.1.7 Modeling

- **Data:** csv files
- **merging_and_analyzing.ipynb:** Merging of all the datasets, EDA
- **correlation_anaylsis.ipynb:** Correlation analysis on the entire dataset, with different feature families included
- **modeling.ipynb:** Training models on the final dataset and getting results, **run this file to run the finalized models and to see the comparison against the betting odds.**

Chapter B
Pseudocode Listings

```
FUNCTION get_json(match_id):
    Open Chrome browser in headless mode
    Go to Sofascore API link for the match
    Retrieve page source
    Parse page with HTML parser
    IF JSON data is found:
        Return JSON dictionary
    ELSE:
        Return empty

FUNCTION get_stats(match_id):
    TRY:
        Request data directly from Sofascore API
    EXCEPT (e.g., connection error):
        Fallback -> use get_json(match_id)

    Extract 'statistics' from JSON
    Convert each statistics group into a small table
    Merge all tables into one DataFrame
    Separate home and away stats into columns
    Return final combined DataFrame

FUNCTION stats_to_dataframe(season, league):
    Use Sofascore API to get list of all matches
    FOR each valid match in season:
        Store metadata (date, id, teams, scores)
        Call get_stats(match_id) to retrieve stats
        Append to stats DataFrame
    RETURN metadata DataFrame, stats DataFrame

FUNCTION stats_to_dataframe_range(start_season, end_season, league):
    FOR each season between start and end:
        Call stats_to_dataframe(season, league)
        Append results into combined metadata and stats DataFrames
    RETURN combined metadata and stats
```

Figure B.1: Pseudocode for Sofascore data acquisition and aggregation pipeline

```
FUNCTION fill_missing_with_regression(features, target_column):
    IF target_column has no missing values:
        RETURN target_column unchanged

    Split data into:
        - Training set (rows without NaNs)
        - Prediction set (rows with NaNs)

    Split training set further into:
        - Fit set
        - Validation set

    Initialize best_score = +infinity
    Initialize best_model = None

    FOR degree in [1, 2, 3]:
        Train polynomial regression model of given degree
        Predict on validation set
        Compute Mean Absolute Error (MAE)
        IF MAE < best_score:
            best_score = MAE
            best_model = current model
            best_degree = degree

    Use best_model to predict missing values
    Fill NaNs in target_column with predictions
    RETURN updated target_column

FOR each feature in dataset:
    IF feature contains NaNs:
        fill_missing_with_regression(features, feature)
    ELSE:
        continue

Drop features where regression MAE is unacceptably high
```

Figure B.2: Pseudocode for regression-based imputation and feature gating in data cleaning

```
FUNCTION AggregationWithErrorAnalysis(df, features, window):
    methods := [linear, mean, EMA, WMA, median]
    sort df by team_id, date
    preallocate result arrays

    FOR each team in df:
        FOR each feature in features:
            FOR each match i:
                past := last 'window' values before i
                compute mean, median, EMA, WMA
                compute linear regression if enough data
                round values if feature is discrete
                store prediction and absolute error

    return df with aggregated values and errors


FUNCTION ErrorAnalysis(df, features, methods):
    collect all error columns
    compute mean error per (feature, method)
    return table [feature, method, mean_error]


FUNCTION RunAggregationWindows(df, features, methods, W):
    FOR each window in W:
        agg_df := AggregationWithErrorAnalysis(df, features, window)
        err_df := ErrorAnalysis(agg_df, features, methods)
        store results with window tag
    return best method per feature per window


FUNCTION MergeAggregatedData(base_df, aggregated_dfs, mapping):
    FOR each feature -> (window, method):
        select corresponding aggregated values
        merge into base_df
    return enriched dataset
```

Figure B.3: Simplified pseudocode for feature aggregation and error analysis pipeline

```
FUNCTION DefineFeatureSet():
    feature_cols := aggregated Sofascore (means/medians)
                    + ClubELO, Seasonal_Elo
                    + team-aggregated FIFA attributes
    return feature_cols

FUNCTION EvaluateModelsTSCV(df, feature_cols, target):
    split chronologically with TimeSeriesSplit
    FOR each fold:
        train & evaluate:
            - Random Forest (n_estimators=300) -> MAE, R2, top-5 features
            - Neural Net (ReLU + Dropout, scaled, early stopping) -> MAE, R2
            - XGBoost (early stopping) -> MAE, R2
        record metrics
    aggregate metrics across folds (mean MAE, mean R2)
    train final models on last fold blocks:
        rf_model, nn_model, xgb_model
    return rf_model, nn_model, xgb_model, perf_summary

FUNCTION BuildSmallModels(df, target, top_features):
    train small Random Forest and small Neural Net on top_features
    evaluate with chronological split; average MAE/R2
    return small_rf, small_nn, small_perf

# --- Orchestration ---
merged_df        := LoadAndPrepareData()
feature_cols     := DefineFeatureSet()
performance_df   := empty table
full_models_dict := {}
target_list      := list of ~20 match stats (e.g., corners, fouls, xG-like proxies,
    saves, game_dominance)

# 1) Full models (all features)
FOR each target IN target_list:
    rf, nn, xgb, perf = EvaluateModelsTSCV(merged_df, feature_cols, target)
    store {rf, nn, xgb} in full_models_dict[target]
    append perf to performance_df

# 2) Derive top features per target (from RF importances)
FOR each target IN target_list:
    top_features_dict[target] := SummariseRFImportances(performance_df, target)

# 3) Small models (top features only)
full_models_dict_small := {}
FOR each target IN target_list:
    small_rf, small_nn, small_perf := BuildSmallModels(merged_df, target,
        top_features_dict[target])
    store {small_rf, small_nn} in full_models_dict_small[target]
    append small_perf to performance_df

# 4) Select best plan per target
best_df   := per target, row with lowest MAE from performance_df
best_plan := map target -> (best_model_name, scope in {"full","small"})

# 5) Unified prediction helpers
FUNCTION GetFeaturesFor(target, scope):
    if scope == "full":  return feature_cols
    if scope == "small": return top_features_dict[target]

FUNCTION GetModelFor(target, model_name, scope):
    if scope == "full":  return full_models_dict[target][model_name]
    if scope == "small": return full_models_dict_small[target][model_name]
```

Figure B.4: Simplified pipeline for ML-based feature aggregation

Chapter C
Tables

Table C.1: Condensed Feature Inventory with Full Source/Type Labels

Feature Type	Feature Source	Feature	DT	Versions	Aggregation Types
Pre-manufactured metrics	Understat	DEEP	T	O, WA, MA	WA: mean(20); MA: NN(s)
Pre-manufactured metrics	Understat	PPDA	T	O, WA, MA	WA: med(20); MA: XGB
Pre-manufactured metrics	Understat	xG	P	O, WA, MA	WA: med(15); MA: NN(s)
In-game metrics	Understat	red_card	P	O, WA, MA	WA: med(8); MA: NN(f)
In-game metrics	Understat	key_passes	P	O, WA, MA	WA: med(15); MA: RF
Pre-manufactured metrics	Understat	xGBuildup	P	O, WA, MA	WA: med(15); MA: NN(s)
In-game metrics	Sofascore	Corner kicks	T	O, WA, MA	WA: mean(17); MA: NN(s)
In-game metrics	Sofascore	Fouls	T	O, WA, MA	WA: mean(20); MA: RF
In-game metrics	Sofascore	Shots off target	T	O, WA, MA	WA: mean(16); MA: RF
In-game metrics	Sofascore	Throw-ins	T	O, WA, MA	WA: mean(15); MA: RF
In-game metrics	Sofascore	Ground duels	T	O, WA, MA	WA: mean(15); MA: NN(s)
In-game metrics	Sofascore	Aerial duels	T	O, WA, MA	WA: mean(20); MA: RF
Self-Generated metrics	Sofascore	Game dominance	T	O, WA, MA	WA: mean(10); MA: XGB
Self-Generated metrics	Sofascore	Blocked per SOT	T	O, WA, MA	WA: med(15); MA: NN(s)
Self-Generated metrics	Sofascore	SOT per total shots	T	O, WA, MA	WA: mean(15); MA: NN(s)
Self-Generated metrics	Sofascore	Hit woodwork / shots	T	O, WA, MA	WA: med(20); MA: NN(f)
Self-Generated metrics	Sofascore	Outside box / shots	T	O, WA, MA	WA: mean(20); MA: NN(s)
In-game metrics	Sofascore	Fouled in final 3rd	T	O, WA, MA	WA: mean(20); MA: XGB
In-game metrics	Sofascore	Tackles won	T	O, WA, MA	WA: mean(10); MA: NN(f)
In-game metrics	Sofascore	Dribbles	T	O, WA, MA	WA: mean(20); MA: NN(s)
In-game metrics	Sofascore	Crosses	T	O, WA, MA	WA: mean(4); MA: RF
In-game metrics	Sofascore	Long balls	T	O, WA, MA	WA: med(20); MA: XGB
In-game metrics	Sofascore	Yellow cards	T	O, WA, MA	WA: mean(20); MA: RF
In-game metrics	Sofascore	Offsides	T	O, WA, MA	WA: med(20); MA: XGB
In-game metrics	Sofascore	Goalkeeper saves	T	O, WA, MA	WA: mean(15); MA: RF
Self-Generated metrics	FIFA	Overall	P	O	—
Self-Generated metrics	FIFA	Finishing_adj	P	O	—
Self-Generated metrics	FIFA	Shot Power_adj	P	O	—
Self-Generated metrics	FIFA	Long Shots_adj	P	O	—
Self-Generated metrics	FIFA	Heading_adj	P	O	—
Self-Generated metrics	FIFA	Curve_adj	P	O	—
Self-Generated metrics	FIFA	FK Acc._adj	P	O	—
Self-Generated metrics	FIFA	Penalties_adj	P	O	—
Self-Generated metrics	FIFA	Short Pass_adj	P	O	—
Self-Generated metrics	FIFA	Long Pass_adj	P	O	—
Self-Generated metrics	FIFA	Stand Tackle_adj	P	O	—
Self-Generated metrics	FIFA	Interceptions_adj	P	O	—
Self-Generated metrics	FIFA	Jumping_adj	P	O	—
Self-Generated metrics	FIFA	Stamina_adj	P	O	—
Self-Generated metrics	FIFA	Strength_adj	P	O	—
Self-Generated metrics	FIFA	Acceleration_adj	P	O	—
Self-Generated metrics	FIFA	Agility_adj	P	O	—
Self-Generated metrics	FIFA	Balance_adj	P	O	—
Self-Generated metrics	FIFA	GK Handling_adj	P	O	—
Self-Generated metrics	FIFA	GK Kicking_adj	P	O	—
Self-Generated metrics	FIFA	GK Positioning_adj	P	O	—
Self-Generated metrics	FIFA	Reactions_adj	P	O	—
Self-Generated metrics	FIFA	Att. Position_adj	P	O	—
Self-Generated metrics	FIFA	Vision_adj	P	O	—
Self-Generated metrics	FIFA	Dribbling_adj	P	O	—
Self-Generated metrics	FIFA	Overall Passing_adj	P	O	—
Self-Generated metrics	FIFA	Overall Shooting_adj	P	O	—
Self-Generated metrics	FIFA	Overall Defence_adj	P	O	—
Self-Generated metrics	FIFA	Overall Physical_adj	P	O	—
Self-Generated metrics	FIFA	Overall Goalkeeper_adj	P	O	—
Pre-manufactured metrics	ClubELO	ClubELO	T	O	—
Self-Generated metrics	Yarkin	Seasonal_Elo	P	O	—

DT: Data Type, T: Team Level, P: Player Level, O: Original, WA: Window Aggregated (with window size), MA: Machine Learning Aggregated, med: Median, NN: Neural Network, (s/f): small/full, RF: Random Forest, XGB: XGBoost